

¿Qué es UML? Versión 2.5 de UML

¿Qué diagramas componen UML?

El **Lenguaje Unificado de Modelado** o UML («Unified Modeling Language») es un lenguaje estandarizado de modelado. Está especialmente desarrollado para ayudar a todos los intervinientes en el desarrollo y modelado de un sistema o un producto software a describir, diseñar, especificar, visualizar, construir y documentar todos los artefactos que lo componen, sirviéndose de varios tipos de diagramas.



Estos diagramas contenidos en UML son la forma más común y más utilizada de modelado de software. Modelar consiste en crear un diseño previo de una aplicación antes de proceder a su desarrollo e implementación, aunque en ocasiones concretas puede hacerse posteriormente. De la misma forma que un arquitecto dibuja y diseña planos sobre el edificio que va a construir, un analista de software (u otros perfiles que cargan con este rol) crea distintos diagramas UML que sirven de base para la posterior construcción/mantenimiento del sistema. El modelado es la principal forma de visualizar el diseño de una aplicación con la finalidad de compararla con los requisitos antes de que el equipo de desarrollo comience a codificar.

El modelado es vital en todo tipo de proyectos, pero cobra especial importancia a medida que el proyecto crece de tamaño. Para que una aplicación funcione correctamente, debe ser diseñada para permitir la **escalabilidad, la seguridad y la ejecución**. Utilizando diagramas UML se consigue visualizar y verificar los diseños de sus sistemas de software antes de que la implementación del código haga que los cambios sean difíciles y demasiado costosos.

Estos **diagramas de UML** son representaciones gráficas que muestran de forma parcial un sistema de información, bien esté siendo desarrollado o ya lo haya sido. Suelen estar acompañados de documentación que les sirve de apoyo, adoptando esta múltiples formas. Además, UML no excluye la posibilidad de mezclar diagramas, algo que, de hecho, suele ser bastante común. Siempre teniendo en cuenta una de las máximas de UML: **Una imagen vale más que mil palabras**.

Como **principal desventaja** ampliamente mencionada de UML podemos nombrar el hecho de que se trata de un lenguaje muy amplio, haciendo, en ocasiones, complicado utilizar todas las posibilidades que ofrece. No obstante, los analistas tienden a utilizar los diagramas de forma sencilla, consiguiendo que sean entendidos fácilmente por cualquier persona que accedan a ellos.

¿Por qué UML?

Los modelos o diagramas de UML nos ayudan a trabajar a un mayor nivel de abstracción. Permite modelar cualquier tipo de aplicación corriendo en cualquier combinación de hardware y software, sistema operativo, lenguaje de programación y red, es decir, UML es independiente de la plataforma hardware sobre la que actúa el software. Su flexibilidad permite modelar cualquier tipo de aplicación e, incluso, otros tipos de proyecto que no son puramente software.

UML ofrece ese modelado utilizando diagramas y se denomina lenguaje por ser una forma común de expresarse por todos los analistas, desarrolladores y usuarios. Está desarrollado para ayudar a todos estos (y más) perfiles a especificar, visualizar, construir y documentar todos los componentes de un proyecto. A pesar de que cada diagrama UML en particular aporta su visión particular al modelado, el lenguaje en su conjunto tiene algunas características que interesa resaltar:

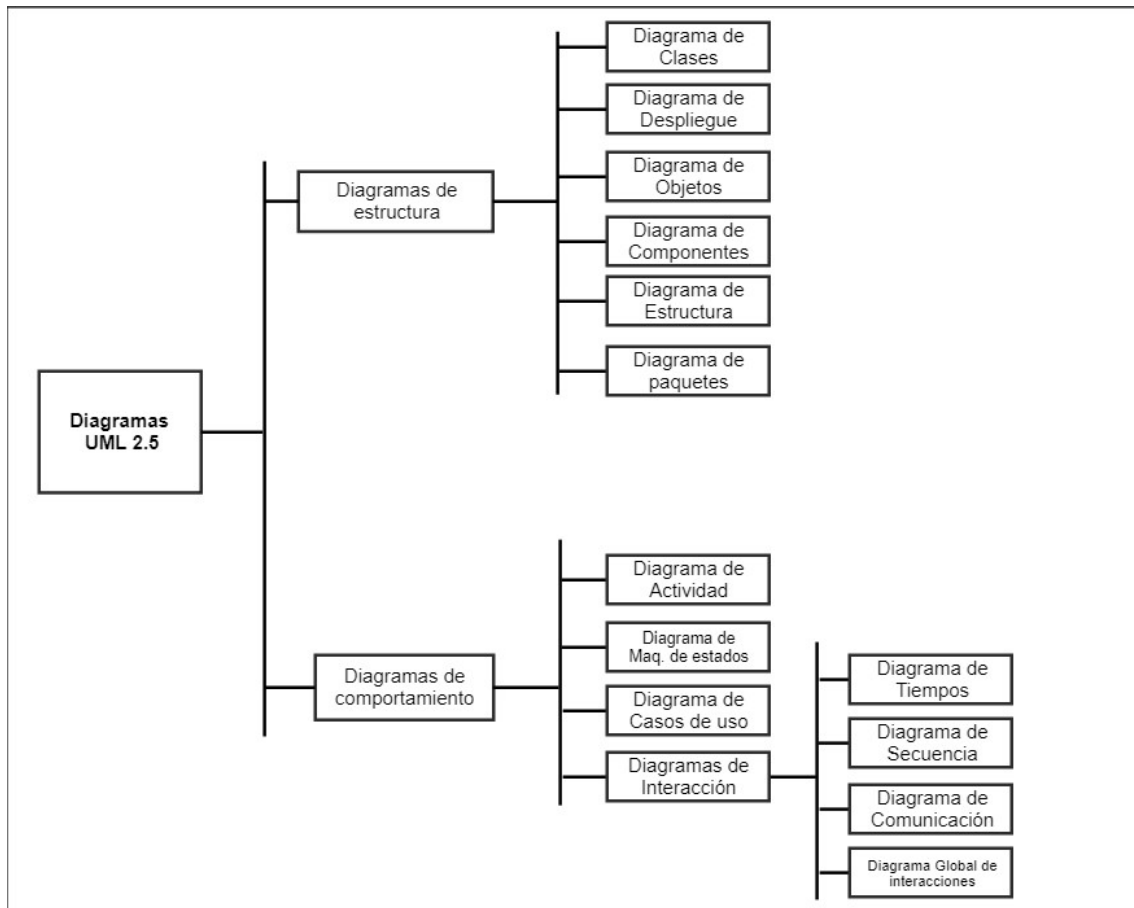
- Es muy **sencillo**. Pese a que si es usado de forma completa puede llegar a complicarse, lo normal es que se simplifique.
- Es capaz de modelar **todo tipo de sistemas**.
- Es un lenguaje **universal**, haciendo que todos los miembros del equipo se relacionen a través de sus diagramas sean del ámbito que sean.
- Es fácilmente **extensible**. Tiene mecanismos sencillos para especializar los conceptos fundamentales.
- Es **visual** y, por lo tanto, intuitivo.
- Es **independiente** del desarrollo, del lenguaje y de la plataforma.
- Bien ejecutado aporta un conjunto considerable de **buenas prácticas**.
- **No está completo**. Utilizando los distintos diagramas no podemos estar seguros de comprender con totalidad el sistema que va a desarrollarse. Los diagramas, para facilitar su comprensión pueden (y suelen) omitir información, pueden tener partes que se entienden de distintas maneras o, incluso, pueden tener conceptos que no pueden ser representados por ningún diagrama.

Tipos de diagramas UML

A día de hoy, en la **versión 2.5.1 de UML**, existen dos clasificaciones de diagramas:

Los **diagramas estructurales** y los **diagramas de comportamiento**.

Todos los diagramas UML están contenidos en esta clasificación.



Clasificación de los diagramas UML

Los diagramas estructurales muestran la estructura estática del sistema y sus partes en diferentes niveles de abstracción. Existen un total de **siete tipos de diagramas de estructura**:

[Diagrama de clases](#)

Muestra la estructura del sistema, subsistema o componente utilizando clases con sus características, restricciones y relaciones: asociaciones, generalizaciones, dependencias, etc.

[Diagrama de componentes](#)

Muestra componentes y dependencias entre ellos. Este tipo de diagramas se utiliza para el desarrollo basado en componentes (CDB), para describir sistemas con arquitectura orientada a servicios (SOA).

[Diagrama de despliegue](#)

Muestra la arquitectura del sistema como despliegue (distribución) de artefactos de software.

[Diagrama de objetos](#)

Un gráfico de instancias, incluyendo objetos y valores de datos. Un diagrama de objeto estático es una instancia de un diagrama de clase; muestra una instantánea del estado detallado de un sistema en un punto en el tiempo.

[Diagrama de paquetes](#)

Muestra los paquetes y las relaciones entre los paquetes.

[Diagrama de perfiles](#)

Diagrama UML auxiliar que permite definir estereotipos personalizados, valores etiquetados y restricciones como un mecanismo de extensión ligero al estándar UML. Los perfiles permiten adaptar el metamodelo UML para diferentes plataformas o dominios.

[Diagrama de estructura compuesta](#)

Muestra la estructura interna (incluidas las partes y los conectores) de un clasificador estructurado.

Diagramas de comportamiento

A diferencia de los diagramas estructurales, muestran como se comporta un sistema de información de forma dinámica. Es decir, describe los cambios que sufre un sistema a través del tiempo cuando está en ejecución. Hay un total de siete diagramas de comportamiento, clasificados de la siguiente forma:

[Diagrama de actividades](#)

Muestra la secuencia y las condiciones para coordinar los comportamientos de nivel inferior, en lugar de los clasificadores que poseen esos comportamientos. Estos son comúnmente llamados modelos de flujo de control y flujo de objetos.

[Diagrama de casos de uso](#)

Describe un conjunto de acciones (casos de uso) que algunos sistemas o sistemas (sujetos) deben o pueden realizar en colaboración con uno o más usuarios externos del sistema (actores) para proporcionar algunos resultados observables y valiosos a los actores u otros interesados del sistema(s).

[Diagrama de máquina de estados](#)

Se utiliza para modelar el comportamiento discreto a través de transiciones de estados finitos. Además de expresar el comportamiento de una parte del sistema, las máquinas de estado también se pueden usar para expresar el protocolo de uso de parte de un sistema.

Diagramas de interacción.

Es un subconjunto de los diagramas de comportamiento. Comprende los siguientes diagramas:

[Diagrama de secuencia](#)

Es el tipo más común de diagramas de interacción y se centra en el intercambio de mensajes entre líneas de vida (objetos).

[Diagrama de comunicación](#)

Se enfoca en la interacción entre líneas de vida donde la arquitectura de la estructura interna y cómo esto se corresponde con el paso del mensaje es fundamental. La secuencia de mensajes se da a través de una numeración.

[Diagrama de tiempos](#)

Se centran en las condiciones que cambian dentro y entre las líneas de vida a lo largo de un eje de tiempo lineal.

Diagrama global de interacciones

Los diagramas global de interacciones brindan una descripción general del flujo de control donde los nodos del flujo son interacciones o usos de interacción.

¿Qué versiones existen de UML?

La versión actual de UML es la 2.5.1 y fue publicada en diciembre de 2017. UML es gestionada y actualizada por la OMG (Object Management Group).

Los creadores originales de UML son 3: **Jim Rumbaugh, Grady Booch e Ivar Jacobson.**

Esta es la lista de versiones que han sido publicadas:

- 1.1 – Noviembre de 1997
- 1.3 – Marzo de 2000
- 1.4 – Septiembre de 2001
- 1.5 – Marzo de 2003
- 1.4.2 – Enero de 2005
- 2.0 – Octubre de 2005
- 2.1 – Abril de 2006
- 2.1.1 – Febrero de 2007
- 2.1.2 – Noviembre de 2007
- 2.2 – Febrero de 2009
- 2.3 – Mayo de 2010
- 2.4.1 – Agosto de 2011
- 2.5 – Junio de 2015
- **2.5.1 – Diciembre de 2017 (Última versión)**

Breve historia de UML

Desde hace unos años, las tecnología de la información y comunicación ya han producido una enorme variedad de métodos y notaciones para llevar a cabo el modelado. Existen métodos y anotaciones para el diseño, la estructura, el procesamiento y el almacenamiento de información. De la misma manera también podemos encontrar métodos para la planificación, modelado, implementación, ensamblaje, prueba, documentación, ajuste, etc. de los sistemas. Entre los conceptos que se utilizan existen algunos relativamente fundamentales y, debido a eso, se expanden más allá del ámbito en el que fueron creados en un principio.

Desde la concepción de la tecnología de la información hasta finales de 1970, los desarrolladores de software se tomaron el desarrollo del software como un arte. Pero estos sistemas fueron poco a poco haciéndose más complejos y por esta razón el mantenimiento y el desarrollo exigía otro tipo de visión, más allá del previamente descrito. Este hecho dio lugar a la ya famosa crisis del software.

Esta crisis lleva al enfoque de ingeniería (ingeniería de software) y la programación estructurada. Se desarrollaron métodos para la estructuración de sistemas y para los procesos de diseño, desarrollo y mantenimiento. Los enfoques orientados a procesos, por ejemplo, el método de salida de procesamiento de entrada de jerarquía, enfatizaron la funcionalidad de los sistemas. Con este método, el sistema total se divide en componentes más pequeños a través de la descomposición funcional.

La crisis del software

Al mismo tiempo, se desarrollaron enfoques orientados a la estructura de datos, como el método de Jackson, en el que la estructura del programa se deriva de la visualización gráfica de las estructuras de datos.

En todos estos métodos y notaciones, dividimos el sistema en dos partes: una sección de datos y una sección de procedimientos. Esto es claramente reconocible en lenguajes de programación más antiguos, como COBOL. Los diagramas de flujo de datos, los diagramas de estructura, los diagramas HIPO y los diagramas de Jackson se utilizan para ilustrar el rango de funciones. Naturalmente, estos primeros métodos enfatizaron el desarrollo de nuevos sistemas.

En la década de 1980, el análisis estructural clásico se desarrolló aún más. Los desarrolladores generaron diagramas de relaciones de entidades para el modelado de datos y redes de Petri para el modelado de procesos.

A medida que los sistemas se volvieron más complejos, ya no se podría diseñar cada sistema «desde cero». Las propiedades, como la mantenibilidad y la reutilización, se hicieron cada vez más importantes. Se desarrollaron lenguajes de programación orientados a objetos, y con ellos, los primeros lenguajes de modelado orientados a objetos surgieron en los años 70 y 80. En la década de 1990, las primeras publicaciones sobre análisis orientado a objetos y diseño orientado a objetos se pusieron a disposición del público. A mediados de la década de 1990, ya existían más de 50 métodos orientados a objetos, así como muchos formatos de diseño. Un lenguaje de modelado unificado parecía indispensable.



A principios de la década de 1990, los métodos orientados a objetos de Grady Booch y James Rumbaugh se utilizaron ampliamente. En octubre de 1994, Rational Software Corporation (parte de IBM desde febrero de 2003) comenzó la creación de un lenguaje de modelado unificado. Primero, acordaron una estandarización de la notación (lenguaje), ya que esto parecía menos elaborado que la estandarización de los métodos. Al hacerlo, integraron el Método Booch de Grady Booch, la Técnica de modelado de objetos (OMT) de James Rumbaugh y la Ingeniería de software orientada a objetos (OOSE), de Ivar Jacobsen, con elementos de otros métodos y publicaron esta nueva notación bajo el nombre UML, versión 0.9.

El objetivo no era formular una notación completamente nueva, sino adaptar, expandir y simplificar los tipos de diagramas existentes y aceptados de varios métodos orientados a objetos, como los diagramas de clase, los diagramas de casos de uso de Jacobson o los diagramas de gráficos de estado de Harel. Los medios de representación que se utilizaron en los métodos estructurados se aplicaron a UML. Por lo tanto, los diagramas de actividad de UML están, por ejemplo, influenciados por la composición de los diagramas de flujo de datos y las redes de Petri.

Lo que es sobresaliente y nuevo en UML no es su contenido, sino su estandarización a un solo lenguaje unificado con un significado definido formalmente.

Compañías conocidas, como IBM, Oracle, Microsoft, Digital, Hewlett-Packard y Unisys se incluyeron en el desarrollo posterior de UML. En 1997, la versión 1.1 de UML fue enviada y aprobada por la OMG. La versión 1.2 de UML, con adaptaciones editoriales, se lanzó en 1998, seguida de la versión 1.3 un año después, y la versión 1.5 de UML en marzo de 2003. Los desarrolladores ya habían estado trabajando en la versión 2.0 de UML desde el año 2000, y se aprobó como una Especificación final adoptada por OMG en junio de 2003. Cuando este libro se imprimió en junio de 2005, la etapa final de adopción por parte de OMG como una especificación disponible aún no se había completado.

Alternativas a UML

Si bien UML es el estándar más utilizado y reconocido para el modelado de sistemas orientados a objetos, existen algunas alternativas que se han desarrollado para abordar diferentes enfoques o necesidades específicas en el modelado de sistemas. Algunas de estas alternativas incluyen:

- SysML (Systems Modeling Language): Diseñado para el modelado de sistemas de ingeniería y sistemas físicos, es una extensión de UML que se centra en el modelado de sistemas complejos.
- BPMN (Business Process Model and Notation): Especializado en modelar procesos de negocios y flujos de trabajo. Aunque no es un reemplazo directo de UML, se utiliza comúnmente en conjunto con él para representar aspectos de procesos de negocio.
- ERD (Entity-Relationship Diagrams): Utilizado principalmente en el modelado de datos y bases de datos. Si bien no reemplaza a UML, se enfoca en la representación de relaciones entre entidades y atributos en un contexto de bases de datos. Puedes aprender más sobre este diagrama a través de [esta entrada en el blog](#).
- Archimate: Un estándar de modelado de arquitectura empresarial que se enfoca en la representación de la arquitectura y la infraestructura empresarial, incluyendo aspectos como procesos, aplicaciones, estructuras de datos, etc.
- Flowchart (Diagrama de Flujo): Aunque más simple en comparación con UML, los diagramas de flujo son útiles para representar algoritmos, flujos de trabajo simples y procesos de toma de decisiones.
- DSL (Domain-Specific Languages): Estos son lenguajes de modelado diseñados específicamente para un dominio particular o un problema específico. A menudo, se utilizan para representar conceptos y abstracciones en un dominio específico de manera más precisa que UML.

Estas alternativas pueden ser utilizadas en situaciones donde UML pueda resultar limitado o donde se necesite un enfoque más especializado para el modelado de sistemas. Sin embargo, es importante destacar que UML sigue siendo el estándar predominante y ampliamente aceptado para el modelado de sistemas de software debido a su versatilidad y amplia gama de diagramas para representar diferentes aspectos de un sistema.

Diagrama de clases

El **diagrama de clases** es uno de los diagramas incluidos en UML 2.5 clasificado dentro de los diagramas de estructura y, como tal, se utiliza para representar los elementos que componen un sistema de información desde un punto de vista estático.

Es importante destacar que, por esta misma razón, este diagrama no incluye la forma en la que se comportan a lo largo de la ejecución los distintos elementos, esa función puede ser representada a través de un diagrama de comportamiento, como por ejemplo un [diagrama de secuencia](#) o un [diagrama de casos de uso](#).

El diagrama de clases es un **diagrama puramente orientado al modelo de programación orientado a objetos**, ya que define las clases que se utilizarán cuando se pase a la fase de construcción y la manera en que se relacionan las mismas. Se podría equiparar, salvando las distancias, al famoso diagrama de modelo Entidad-Relación (E/R), no recogido en UML, tiene una utilidad similar: la representación de datos y su interacción. Ambos diagramas muestran el modelo lógico de los datos de un sistema.

Elementos de un diagrama de clases

El diagrama UML de clases está formado por dos elementos: clases, relaciones e interfaces.

Clases

Las clases son el elemento principal del diagrama y representa, como su nombre indica, una clase dentro del paradigma de la orientación a objetos. Este tipo de elementos normalmente se utilizan para representar conceptos o entidades del «negocio». Una clase define un **grupo de objetos** que comparten características, condiciones y significado. La manera más rápida para encontrar clases sobre un enunciado, sobre una idea de negocio o, en general, sobre un tema concreto es buscar los **sustantivos** que aparecen en el mismo. Por poner algún ejemplo, algunas clases podrían ser: Animal, Persona, Mensaje, Expediente... Es un concepto muy amplio y **resulta fundamental identificar de forma efectiva estas clases**, en caso de no hacerlo correctamente se obtendrán una serie de problemas en etapas posteriores, teniendo que volver a hacer el análisis y perdiendo parte o todo el trabajo que se ha hecho hasta ese momento.

Bajando de nivel una clase está compuesta por tres elementos: **nombre de la clase, atributos, funciones**. **Estos elementos se incluyen en la representación** (o no, dependiendo del nivel de análisis).

Para representar la clase con estos elementos se utiliza una caja que es dividida en tres zonas utilizando para ello líneas horizontales:

- La primera de ellas se utiliza para el **nombre de la clase**. En caso de que la clase sea abstracta se utilizará su nombre en cursiva.
- La segunda, por otra parte, se utiliza para escribir los **atributos** de la clase, uno por línea y utilizando el siguiente formato:

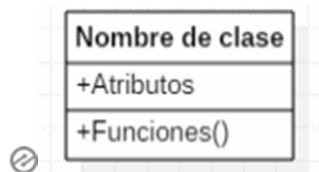
visibilidad nombre_atributo : tipo = valor-inicial { propiedades }

Aunque esta es la forma «oficial» de escribirlas, es común simplificando únicamente poniendo el nombre y el tipo o únicamente el nombre.

- La última de las zonas incluye cada una de las **funciones** que ofrece la clase. De forma parecida a los atributos, sigue el siguiente formato:

visibilidad nombre_funcion { parametros } : tipo-devuelto { propiedades }

De la misma manera que con los atributos, se suele simplificar indicando únicamente el nombre de la función y, en ocasiones, el tipo devuelto.



Notación de una clase

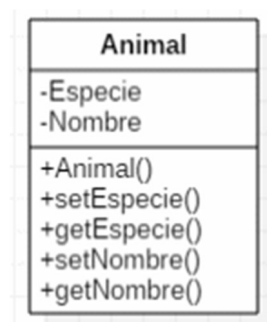
Tanto los atributos como las funciones incluyen al principio de su descripción la visibilidad que tendrá. Esta visibilidad se identifica escribiendo un símbolo y podrá ser:

- **(+) Pública.** Representa que se puede acceder al atributo o función desde cualquier lugar de la aplicación.
- **(-) Privada.** Representa que se puede acceder al atributo o función únicamente desde la misma clase.
- **(#) Protegida.** Representa que el atributo o función puede ser accedida únicamente desde la misma clase o desde las clases que hereden de ella (clases derivadas).



Estos tres tipos de visibilidad son los más comunes. No obstante, pueden incluirse otros en base al lenguaje de programación que se esté usando (no es muy común). Por ejemplo: (/) Derivado o (~) Paquete.

Un ejemplo de clase podría ser el siguiente:

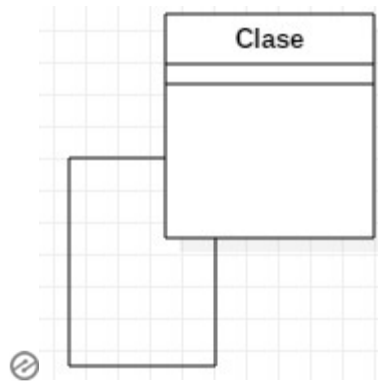


Ejemplo de una clase

En caso de que un atributo o función sea estático, se representa en el diagrama subrayando su nombre. Una característica estática se define como aquella que es compartida por cada clase y no instanciada para cada uno de los objetos de esa clase. Es un concepto muy común.

Relaciones

Una relación **identifica una dependencia**. Esta dependencia puede ser entre dos o más clases (más común) o una clase hacia sí misma (menos común, pero existen), este último tipo de dependencia se denomina *dependencia reflexiva*. Las relaciones se representan con una línea que une las clases, esta línea variará dependiendo del tipo de relación



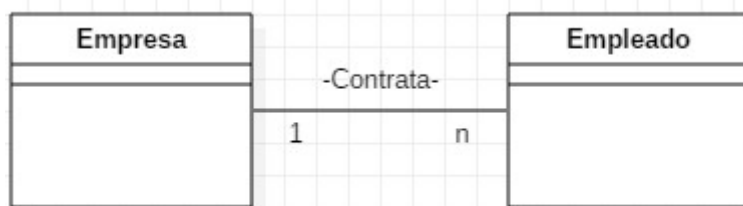
Relación reflexiva

Las relaciones en el diagrama de clases tienen varias propiedades, que dependiendo la profundidad que se quiera dar al diagrama se representarán o no. Estas propiedades son las siguientes:

- **Multiplicidad.** Es decir, el número de elementos de una clase que participan en una relación. Se puede indicar un número, un rango... Se utiliza *n* o *** para identificar un número cualquiera.



- **Nombre de la asociación.** En ocasiones se escriba una indicación de la asociación que ayuda a entender la relación que tienen dos clases. Suelen utilizarse verbos como por ejemplo: «Una empresa contrata a *n* empleados»



Ejemplo de relación Empresa-

Empleado

Tipos de relaciones

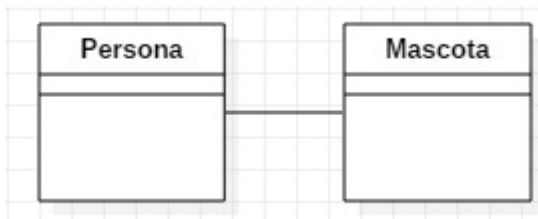
Un diagrama de clases incluye los siguientes tipos de relaciones:

- **Asociación.**
- **Agregación.**
- **Composición.**
- **Dependencia.**
- **Herencia.**

Asociación

Este tipo de relación es el más común y se utiliza para representar dependencia semántica. Se representa con una simple línea continua que une las clases que están incluidas en la asociación.

Un ejemplo de asociación podría ser: «Una mascota pertenece a una persona».



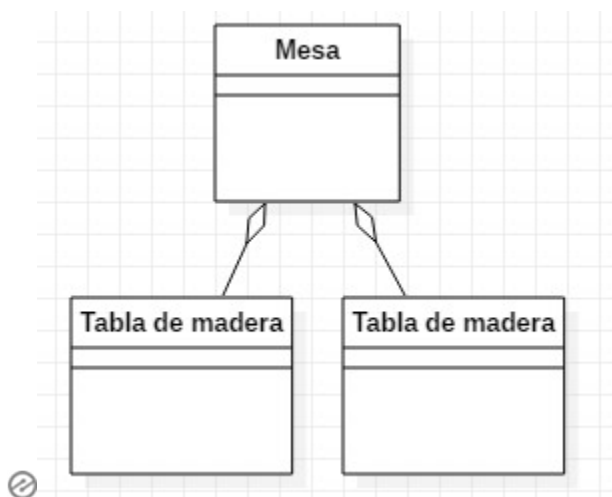
Ejemplo de asociación

Agregación

Es una representación jerárquica que indica a un objeto y las partes que componen ese objeto. Es decir, representa relaciones en las que **un objeto es parte de otro**, pero aun así debe tener **existencia en sí mismo**.

Se representa con una línea que tiene un rombo en la parte de la clase que es una agregación de la otra clase (es decir, en la clase que contiene las otras).

Un ejemplo de esta relación podría ser: «Las mesas están formadas por tablas de madera y tornillos o, dicho de otra manera, los tornillos y las tablas forman parte de una mesa». Como ves, el tornillo podría formar parte de más objetos, por lo que interesa especialmente su abstracción en otra clase.



Ejemplo de agregación

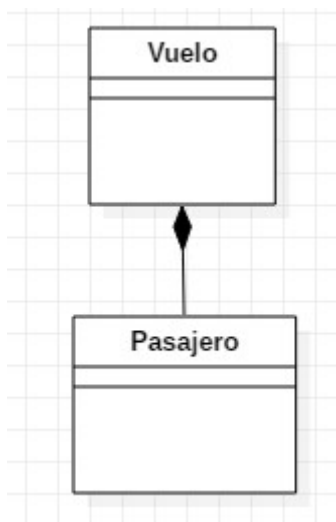
Composición

La composición es similar a la agregación, representa una **relación jerárquica entre un objeto y las partes que lo componen, pero de una forma más fuerte**. En este caso, los elementos que forman parte no tienen sentido de existencia cuando el primero no existe. Es decir, cuando el elemento que contiene los otros desaparece, deben desaparecer todos ya que no tienen sentido por sí mismos sino que dependen del elemento que componen. Además, suelen tener los mismos tiempo de vida. Los componentes no se comparten entre varios elementos, esta es otra de las diferencias con la agregación.



Se representa con una línea continua con un rombo relleno en la clase que es compuesta.

Un ejemplo de esta relación sería: «Un vuelo de una compañía aérea está compuesto por pasajeros, que es lo mismo que decir que un pasajero está asignado a un vuelo»



Ejemplo de composición



Diferencia entre agregación y composición

La diferencia entre agregación y composición es semántica, por lo que **a veces no está del todo definida**. Ninguna de las dos tienen análogos en muchos lenguajes de programación (como por ejemplo Java).

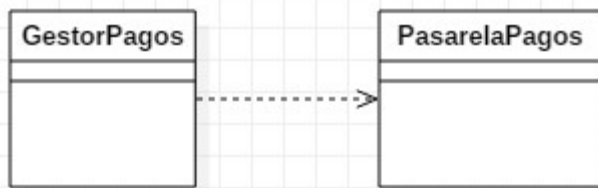
Un «agregado» representa **un todo que comprende varias partes**; de esta manera, un Comité es un agregado de sus Miembros. Una reunión es un agregado de una agenda, una sala y los asistentes. En el momento de la implementación, esta relación no es de contención. (Una reunión no contiene una sala). Del mismo modo, las partes del agregado podrían estar haciendo otras cosas en otras partes del programa, por lo que podrían ser referenciadas por varios objetos que nada tienen que ver. En otras palabras, no existe una diferencia de nivel de implementación entre la agregación y una simple relación de «usos». En ambos casos, un objeto tiene referencias a otros objetos. Aunque no existe una diferencia en la implementación, definitivamente vale la pena capturar la relación en el diagrama UML, tanto porque ayuda a comprender mejor el modelo de dominio, como porque puede haber problemas de implementación que pueden pasar desapercibidos. Podría permitir relaciones de acoplamiento más estrictas en una agregación de lo que haría con un simple «uso», por ejemplo.

La composición, por otro lado, implica **un acoplamiento aún más estricto que la agregación**, y definitivamente implica la contención. El requisito básico es que, si una clase de objetos (llamado «contenedor») se compone de otros objetos (llamados «elementos»), entonces los elementos aparecerán y también serán destruidos como un efecto secundario de crear o destruir el contenedor. Sería raro que un elemento no se declare como privado. Un ejemplo podría ser el nombre y la dirección del Cliente. Un cliente sin nombre o dirección no tiene valor. Por la misma razón, cuando se destruye al cliente, no tiene sentido mantener el nombre y la dirección. (Compare esta situación con la agregación, donde destruir al Comité no debe causar la destrucción de los miembros, ya que pueden ser miembros de otros Comités).

Dependencia

Se utiliza este tipo de relación para **representar que una clase requiere de otra para ofrecer sus funcionalidades**. Es muy sencilla y se representa con una flecha discontinua que va desde la clase que necesita la utilidad de la otra flecha hasta esta misma.

Un ejemplo de esta relación podría ser la siguiente:

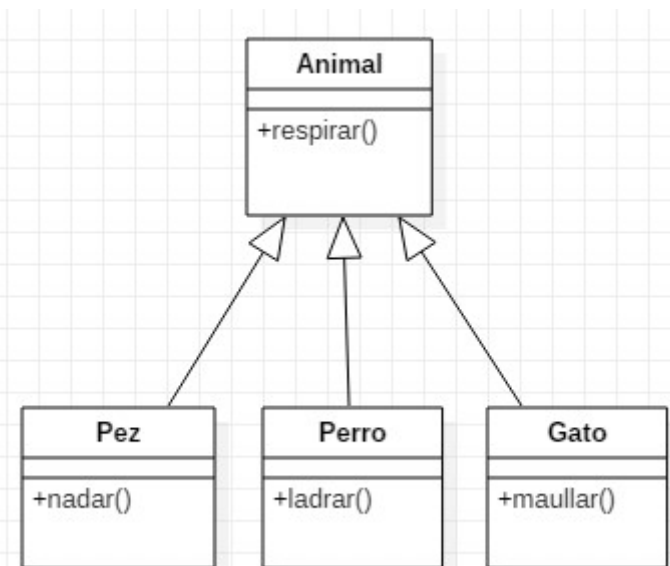


Ejemplo de dependencia

Herencia

Otra relación muy común en el diagrama de clases es la herencia. Este tipo de relaciones permiten que **una clase (clase hija o subclase) reciba los atributos y métodos de otra clase (clase padre o superclase)**. Estos atributos y métodos recibidos se suman a los que la clase tiene por sí misma. Se utiliza en relaciones «es un».

Un ejemplo de esta relación podría ser la siguiente: Un pez, un perro y un gato son animales.



Ejemplo de herencia

En este ejemplo, las tres clases (Pez, Perro, Gato) podrán utilizar la función respirar, ya que lo heredan de la clase animal, pero solamente la clase Pez podrá nadar, la clase Perro ladrar y la clase Gato maullar. La clase Animal podría plantearse ser definida abstracta, aunque no es necesario.

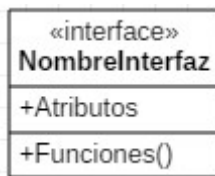
Interfaces

Una interfaz es una entidad que declara una **serie de atributos, funciones y obligaciones**. Es una especie de contrato donde toda instancia asociada a una interfaz debe de implementar los servicios que indica aquella interfaz.

Dado que únicamente son declaraciones **no pueden ser instanciadas**.

Las interfaces se asocian a clases. Una asociación entre una clase y una interfaz representa que esa clase cumple con el contrato que indica la interfaz, es decir, incluye aquellas funciones y atributos que indica la interfaz.

Su representación es similar a las clases, pero indicando arriba la palabra **<<interface>>**.



Notación de interfaz

Cómo dibujar un diagrama de clases

Los diagramas de clase van de la mano con el diseño orientado a objetos. Por lo tanto, saber lo básico de este tipo de diseño es una parte clave para poder dibujar diagramas de clase eficaces.

Este tipo de diagramas son solicitados cuando se está describiendo la vista estática del sistema o sus funcionalidades. Unos pequeños pasos que puedes utilizar de guía para construir estos diagramas son los siguientes:

- **Identifica** los nombres de las clase
El primer paso es identificar los objetos primarios del sistema. Las clases suelen corresponder a sustantivos dentro del dominio del problema.
- **Distingue** las relaciones
El siguiente paso es determinar cómo cada una de las clases u objetos están relacionados entre sí. Busca los puntos en común y las abstracciones entre ellos; esto te ayudará a agruparlos al dibujar el diagrama de clase.
- **Crea** la estructura
Primero, agrega los nombres de clase y vincúlalos con los conectores apropiados, prestando especial atención a la cardinalidad o las herencias. Deja los atributos y funciones para más tarde, una vez que esté la estructura del diagrama resuelta.

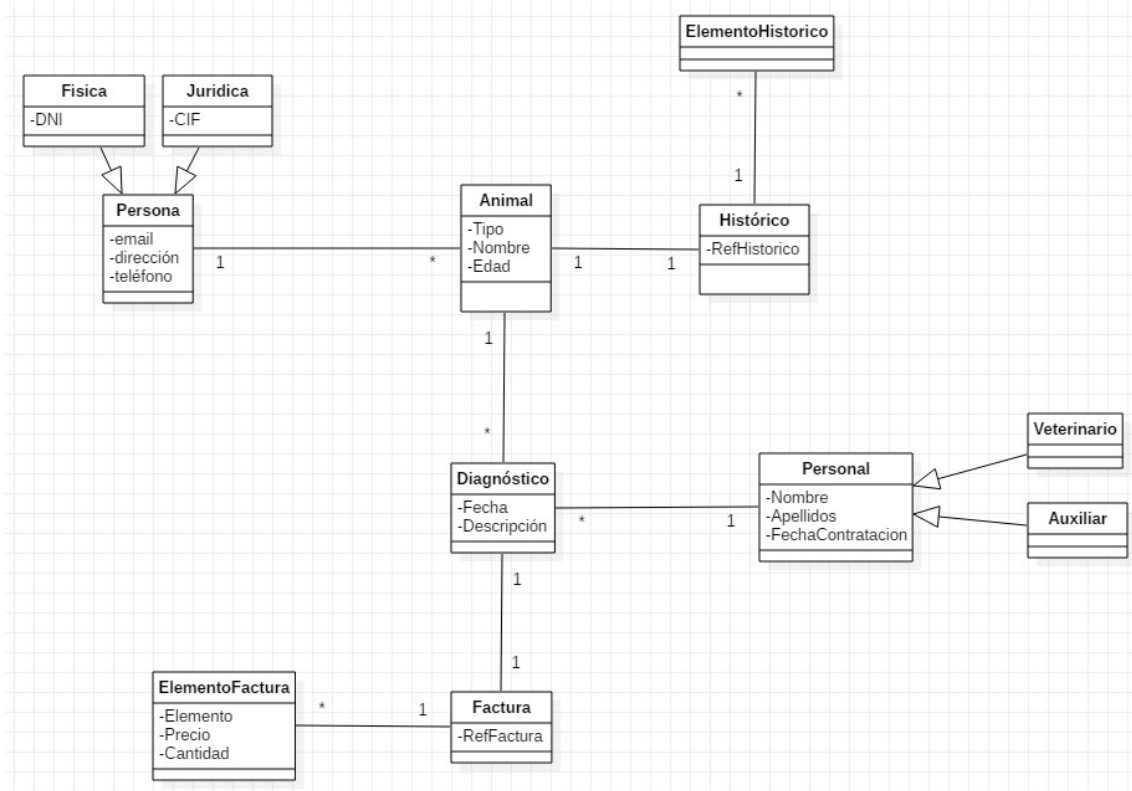
Buenas prácticas en la construcción del diagrama de clases

Te recomendamos seguir estas indicaciones o consejos, que, aunque no son obligatorios, harán que tus diagramas de clases sean de mayor utilidad:

- Los diagramas de clase **pueden tender a volverse incoherentes** a medida que se expanden y crecen. Es mejor evitar la creación de diagramas grandes y **dividirlos** en otros más pequeños que se puedan vincular entre sí más adelante.
- Usando la notación de clase simple, puedes crear rápidamente **una visión general de alto nivel** de su sistema. Se puede crear un diagrama detallado por separado según sea necesario, e incluso vincularlo al primero para una referencia fácil.
- Cuantas más líneas se superpongan en sus diagramas de clase, más abarrotado se vuelve y, por tanto, más se complica utilizarlo. El lector se confundirá tratando de encontrar el camino. Asegúrate de que **no haya dos líneas cruzadas** entre sí, a no ser que no haya más remedio.
- Usa **colores** para agrupar módulos comunes. Diferentes colores en diferentes clases ayudan al lector a diferenciar entre los diversos grupos.

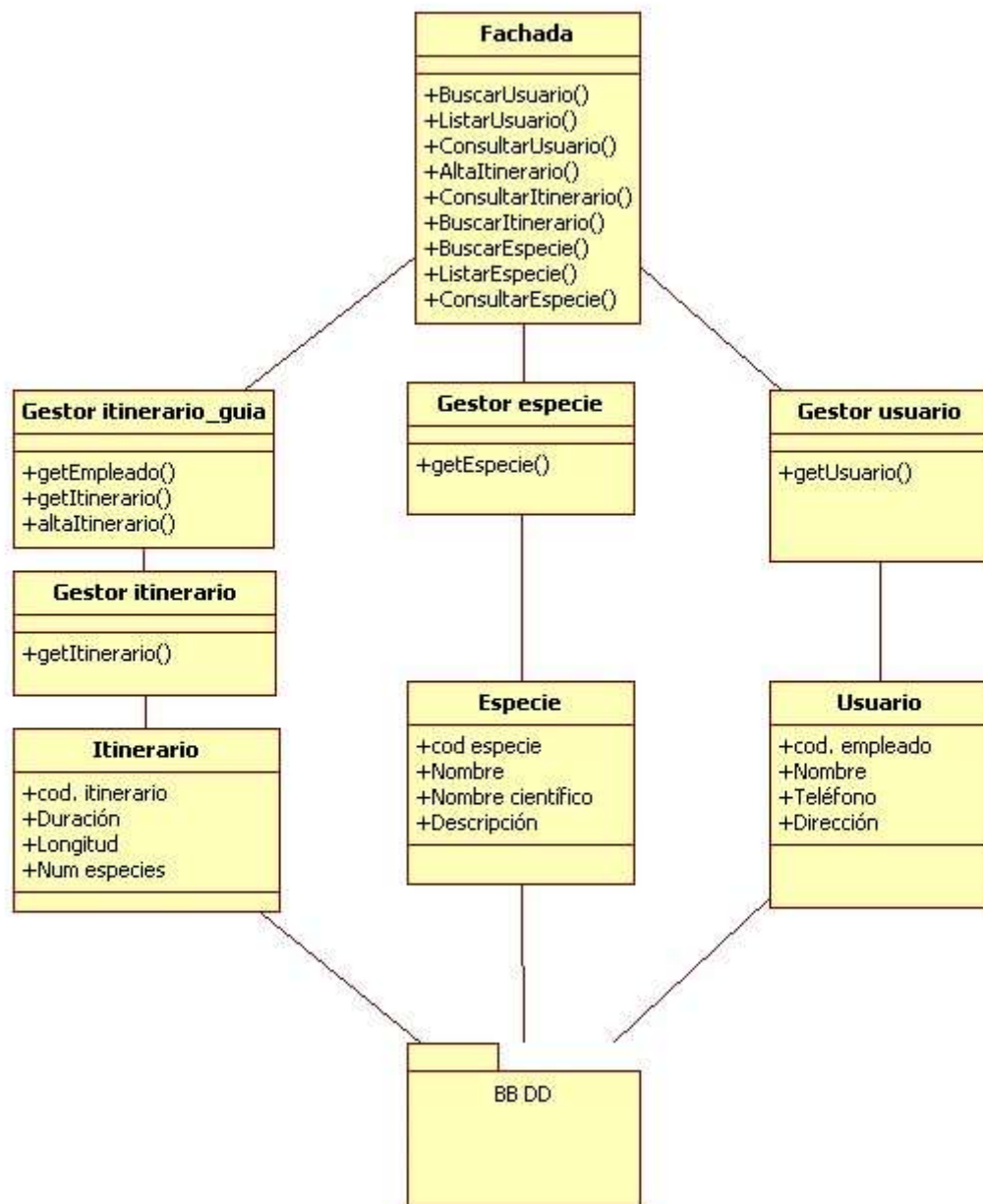
Ejemplos de diagrama de clases

Diagrama de clases clínica veterinaria



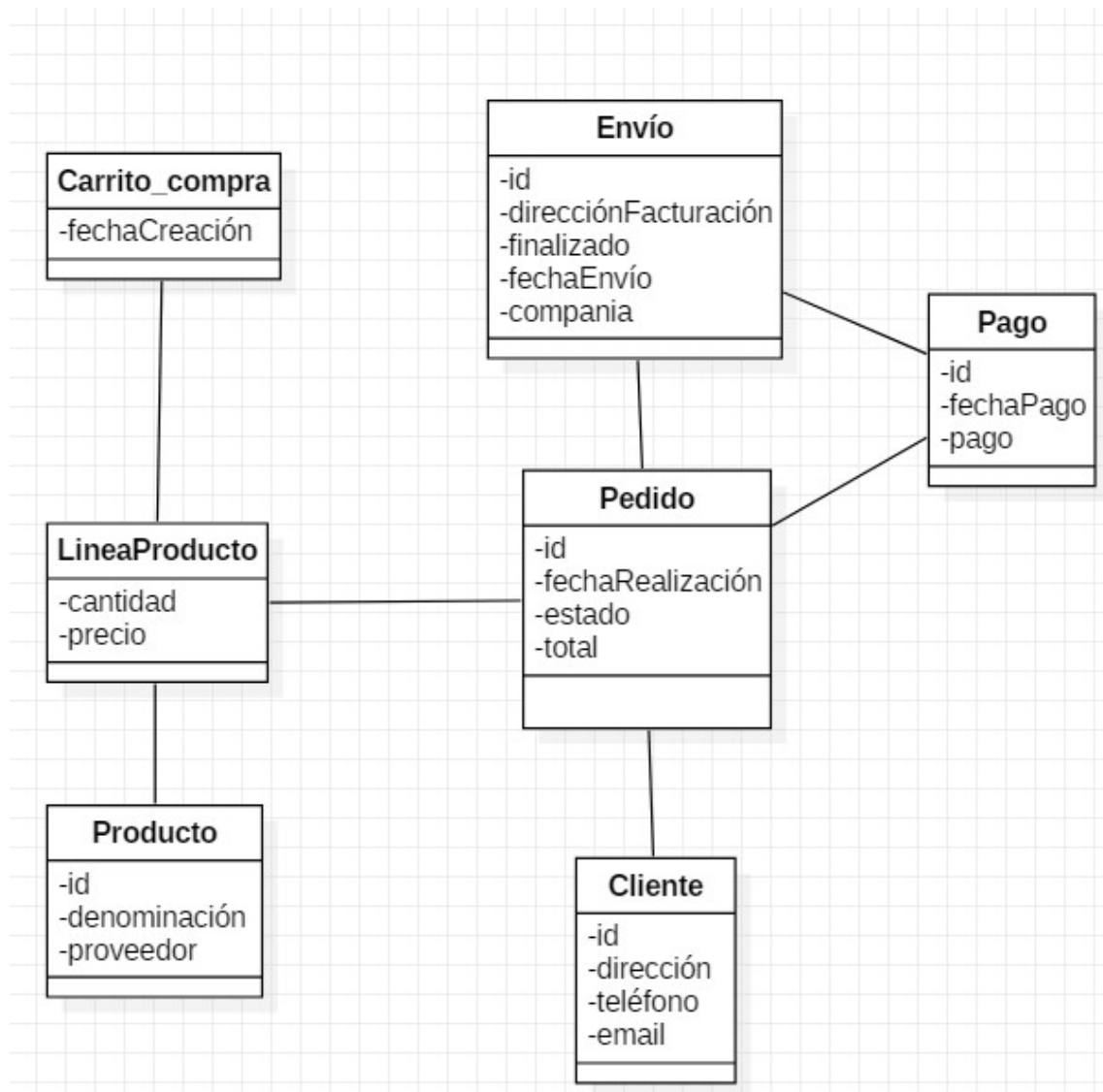
Ejemplo diagrama de clases

Diagrama de clases zoológico



Ejemplo 2 de diagrama de clases

Diagrama de clases de una tienda



Ejemplo 3 diagrama de clases

Diagrama de clases gestión de biblioteca

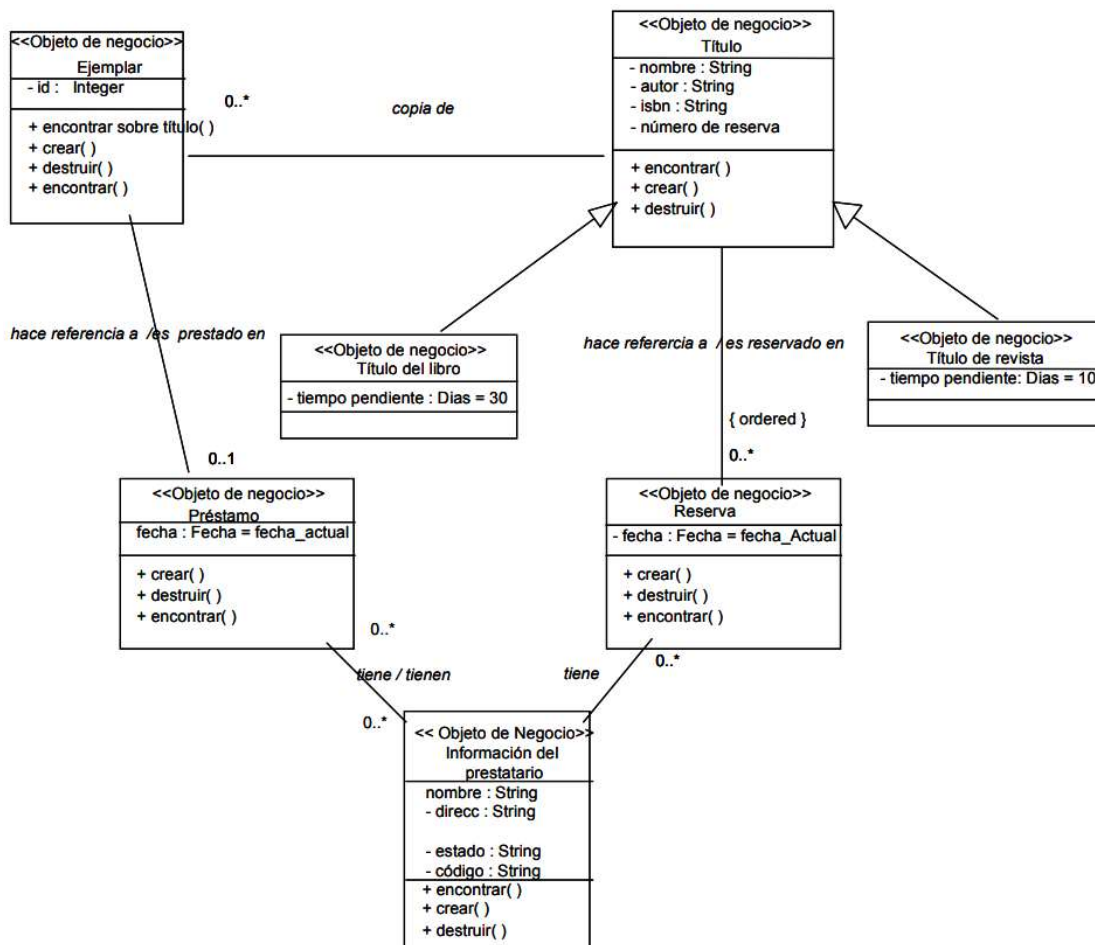
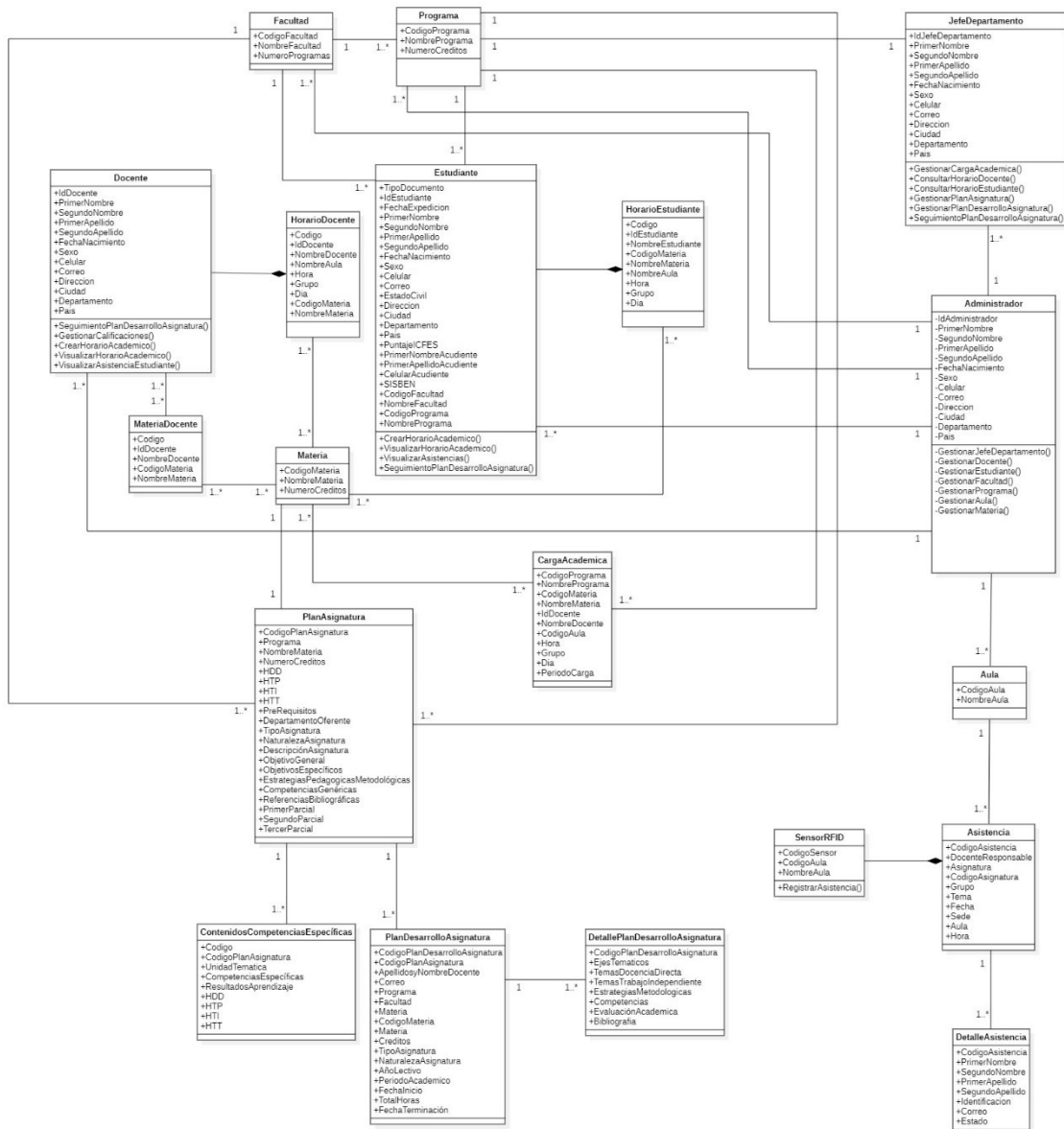


Diagrama de clases gestión de biblioteca (fuente: Métricav3 «Ministerio de Administraciones Públicas»)

Diagrama de clases centro educativo



Ejemplo diagrama de clases centro educativo

Ejemplo de clases para un diagrama de clases de una tienda web

Aquí te dejo un ejemplo de clases con sus atributos que podrías incluir en un diagrama de clases de una tienda online:

1. Usuario:
 - idUsuario: Identificador único del usuario.
 - nombre: Nombre completo del usuario.
 - correoElectronico: Dirección de correo electrónico del usuario.
 - contraseña: Contraseña del usuario.
 - dirección: Dirección de envío del usuario.
 - métodoDePago: Método de pago preferido por el usuario.

2. Producto:
 - idProducto: Identificador único del producto.
 - nombre: Nombre del producto.
 - descripción: Descripción detallada del producto.
 - precio: Precio del producto.
 - stock: Cantidad de unidades disponibles en el inventario.
3. Carrito de Compras:
 - idCarrito: Identificador único del carrito de compras.
 - productos: Lista de productos que el usuario ha añadido al carrito.
 - subtotal: Monto total del carrito antes de aplicar impuestos y descuentos.
 - impuestos: Monto total de impuestos aplicados al carrito.
4. Orden de compra:
 - idOrden: Identificador único de la orden de compra.
 - productos: Lista de productos comprados en la orden.
 - subtotal: Monto total de la orden antes de aplicar impuestos y descuentos.
 - impuestos: Monto total de impuestos aplicados a la orden.
 - envío: Monto del costo de envío de la orden.
 - total: Monto total de la orden incluyendo impuestos, descuentos y costo de envío.
5. Categoría:
 - idCategoría: Identificador único de la categoría.
 - nombre: Nombre de la categoría.
6. Comentarios:
 - idComentario: Identificador único del comentario.
 - producto: Identificador del producto al que se refiere el comentario.
 - usuario: Identificador del usuario que escribió el comentario.
 - comentario: Contenido del comentario.
 - fecha: Fecha de creación del comentario.

El diagrama de despliegue es otro de los diagramas de estructura del conjunto de los diagramas de UML 2.5. Es utilizado para representar la distribución física (estática) de los componentes software en los distintos nodos físicos de la red.

Suele ser utilizado junto con el [diagrama de componentes](#) (incluso a veces con el [diagrama de paquetes](#)) de forma que, juntos, dan una visión general de como estará desplegado el sistema de información. El diagrama de componentes muestra que componentes existen y como se relacionan mientras que el diagrama de despliegue es utilizado para ver como se sitúan estos componentes lógicos en los distintos nodos físicos.

Como prácticamente todos los diagramas de UML, puede ser utilizado para representar aspectos generales o muy específicos, siendo utilizado de forma más común para aspectos generales.

Sus principales características son las siguientes:

- Permite **identificar los nodos** en los que trabajará o utilizarán el sistema de información, identificando a su vez agentes externos e internos que interactuen con el sistema.
- Permite representar de forma clara la **arquitectura física de la red**, así como la distribución del componente software. UML no tiene un tipo de diagramas específico para mostrar la arquitectura de la red, así que se utiliza este tipo de diagrama que cumple efectivamente este cometido, aunque se le suele hacer alguna modificación gráfica.
- Lo más normal es utilizarlo para dar una **visión global**, pero es posible utilizarlo para representar partes específicas de la implementación.

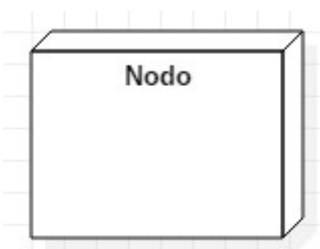
Notación

El diagrama de componentes utiliza, principalmente, dos tipos de elementos: Nodos y conexiones.

Nodos

Los nodos se definen como elementos utilizados para representar un **elemento físico que interactúa** de alguna manera con el sistema o bien forma parte del mismo.

Se representa utilizando un cubo tridimensional, tal y como representa la siguiente figura:



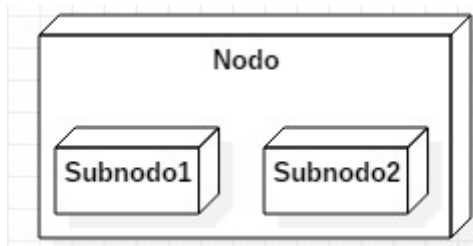
Notación de un nodo

Algunos ejemplos de nodos podrían ser los siguientes: Servidor web, Servidor DNS, Servidor de Aplicaciones, PC Usuario, Base de datos... Como ves todos son elementos físicos que participan de alguna manera en el funcionamiento del sistema.

Los nodos también pueden ser representados utilizando **iconos personalizados** con la finalidad de clarificar el contenido del diagrama. Algunos de estos iconos de uso extendido son:

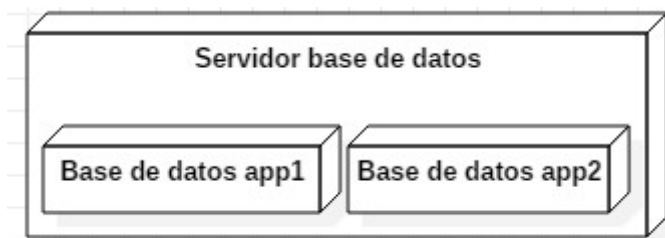
- Un muro para representar un Firewall.
- Un icono de un PC para representar el equipo de un usuario.
- Un circulo con flechas para identificar a un router.
- Una nube para representar una WAN (aunque no es propiamente un nodo)
- Un cilindro para representar una base de datos.

Un nodo a su vez puede tener nodos incluidos en su interior, dando a conocer que son sistemas separados incluidos dentro del mismo nodo físico. De esta forma se compondrían los **nodos compuestos**.



Notación de un nodo con subnodos

Por ejemplo, un nodo llamado Servidor de base de datos podría tener en su interior dos bases de datos separadas de sistemas de información distintos. Podría ser representado de la siguiente forma:

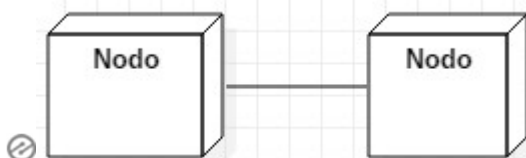


Ejemplo de nodo compuesto

Conexión

La conexión representa una **asociación entre dos nodos**, a través de la cual estos nodos son capaces de transmitir información en forma de mensajes o señales.

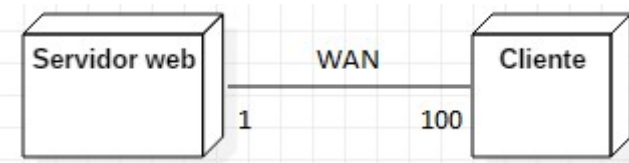
Se representa utilizando una línea continua que une los dos nodos que se asocian.



Notación de una conexión

Es común incluir en las conexiones una etiqueta que represente a través de que **medio se realiza la conexión**. Por ejemplo: Internet, WAN...

También, si es relevante, se suele poner al lado de los nodos el **número de nodos** que participan en la asociación. Por ejemplo, un servidor web al que se conectan usuarios a través de una red WAN y que se prevé una conexión de 100 usuarios tendría la siguiente representación:



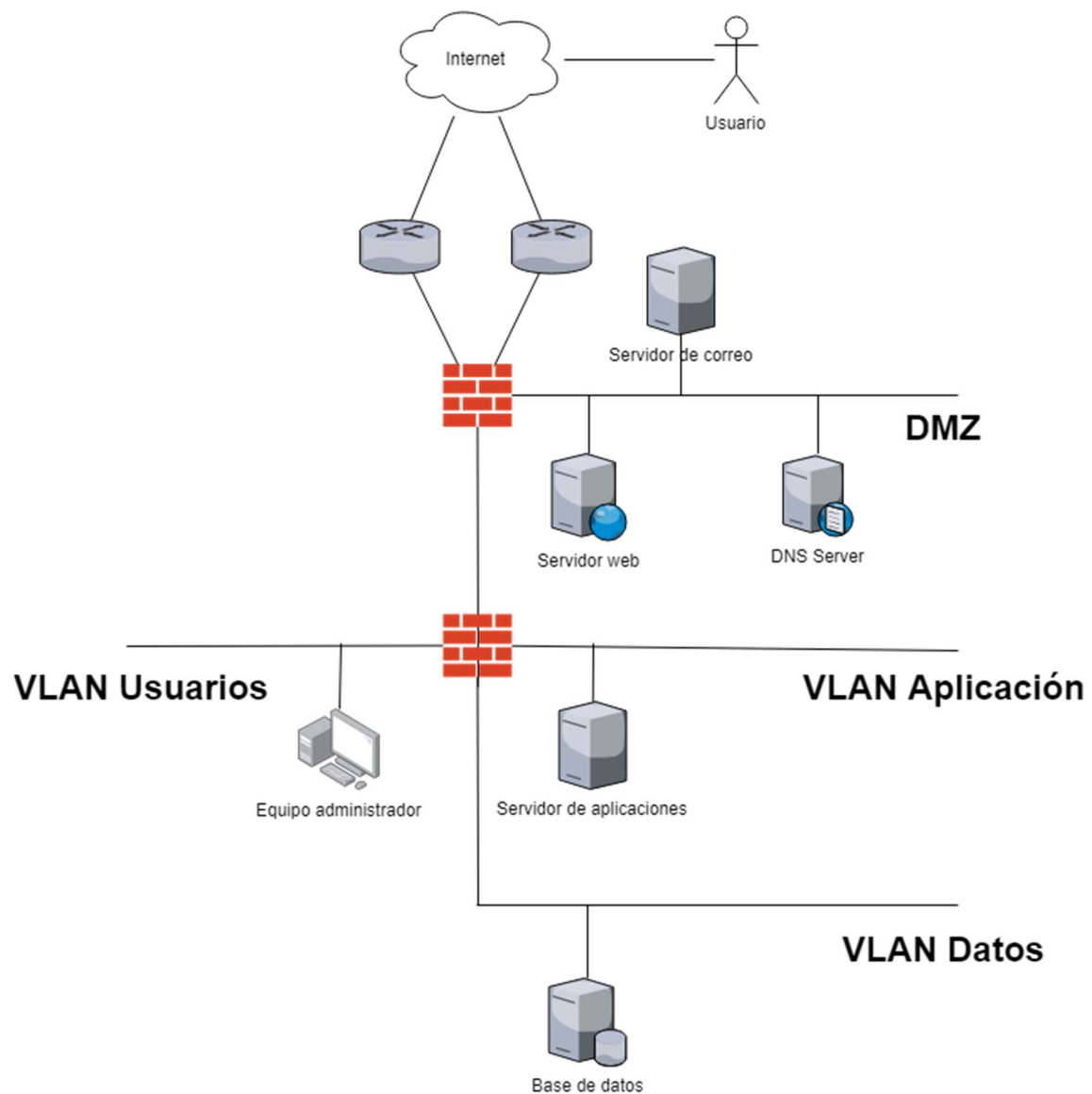
Notación de conexión Cliente-Servidor



Diagramas de arquitectura de red

Como ya se ha mencionado, el estándar UML no tiene un tipo de diagrama para describir la arquitectura de red y, por lo tanto, no proporciona elementos específicos relacionados con la red. Los diagramas de despliegue podría usarse para este propósito, generalmente **con algunas modificaciones** adicionales. El diagrama de arquitectura de red generalmente mostrará nodos de red y rutas de comunicación entre ellos.

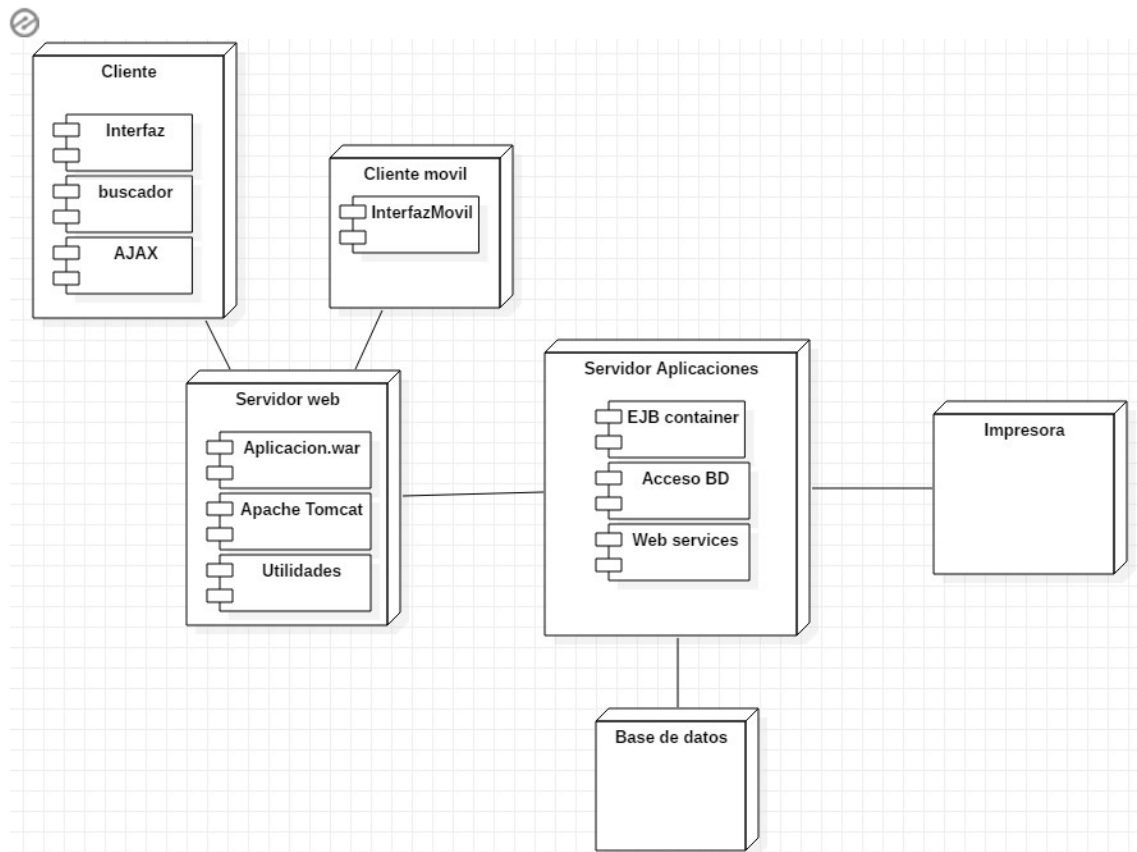
Este es un ejemplo de diagrama de despliegue que actúa como diagrama de red. Como ves, se utilizan distintos iconos para que se entienda mejor:



Ejemplo de diagrama de red

Ejemplo de diagrama de despliegue

La siguiente muestra un ejemplo sobre este tipo de diagrama donde se muestran un total de 6 ndos y algunos de sus componentes:



Ejemplo de diagrama de despliegue

Diagrama de paquetes

El diagrama de paquetes es uno de los **diagramas estructurales** comprendidos en UML 2.5, por lo que, como tal, representa de forma estática los componentes del sistema de información que está siendo modelado. Es utilizado para **definir los distintos paquetes a nivel lógico** que forman parte de la aplicación y la dependencia entre ellos. Es principalmente utilizado por desarrolladores y analistas.

Es importante destacar que este diagrama es utilizado en los sistemas de información con **programación orientada a objetos**. El objetivo principal del diseño debe maximizar la cohesión y minimizar el acoplamiento.

En sí el diagrama es muy sencillo, dependiendo su complicación del detalle con el que se quieran tratar los elementos que mostrará, que puede llegar a ser muy específico.

Elementos de un diagrama de paquetes

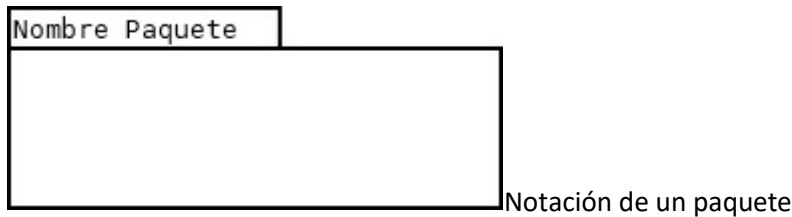
El diagrama de paquetes está constituido por dos elementos: Los paquetes y las dependencias.

Paquete

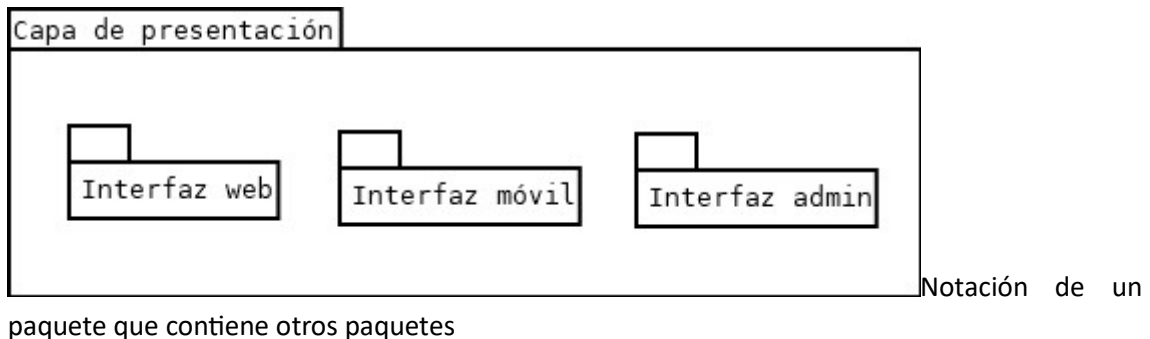
Es el elemento clave del diagrama y que da el nombre al mismo. Un paquete es un **conjunto de elementos**. En concreto puede ser un conjunto de clases, casos de uso, componentes u otros paquetes. No obstante, lo más común es que incluya otros paquetes.

Lo ideal es que este conjunto de elementos tenga una función diferenciada del resto de elementos. De esta forma, además de maximizar la cohesión, se dará la máxima claridad al diagrama y, por tanto, al Sistema de Información. Es también importante identificar con nombres representativos de estas funciones a los distintos paquetes. Por supuesto, hay que tener en cuenta el nivel de granularidad del diagrama.

Se representa con un símbolo simulando una carpeta, con el nombre en la parte superior:



Como ejemplo, un paquete que contiene otros paquetes tendría la siguiente representación:



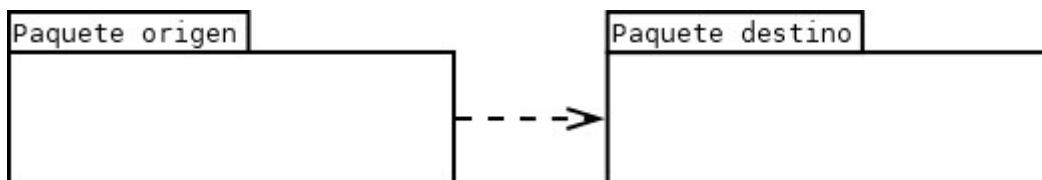
En este caso sería un paquete denominado «Capa de presentación» que contiene los paquetes «Interfaz web», «Interfaz móvil» e «Interfaz admin». En este caso el contenido son otros paquetes, pero podría ser, como ya hemos dicho, otro diagrama.

Por ejemplo, es muy común un paquete que contiene un diagrama de clases.

Dependencia entre paquetes

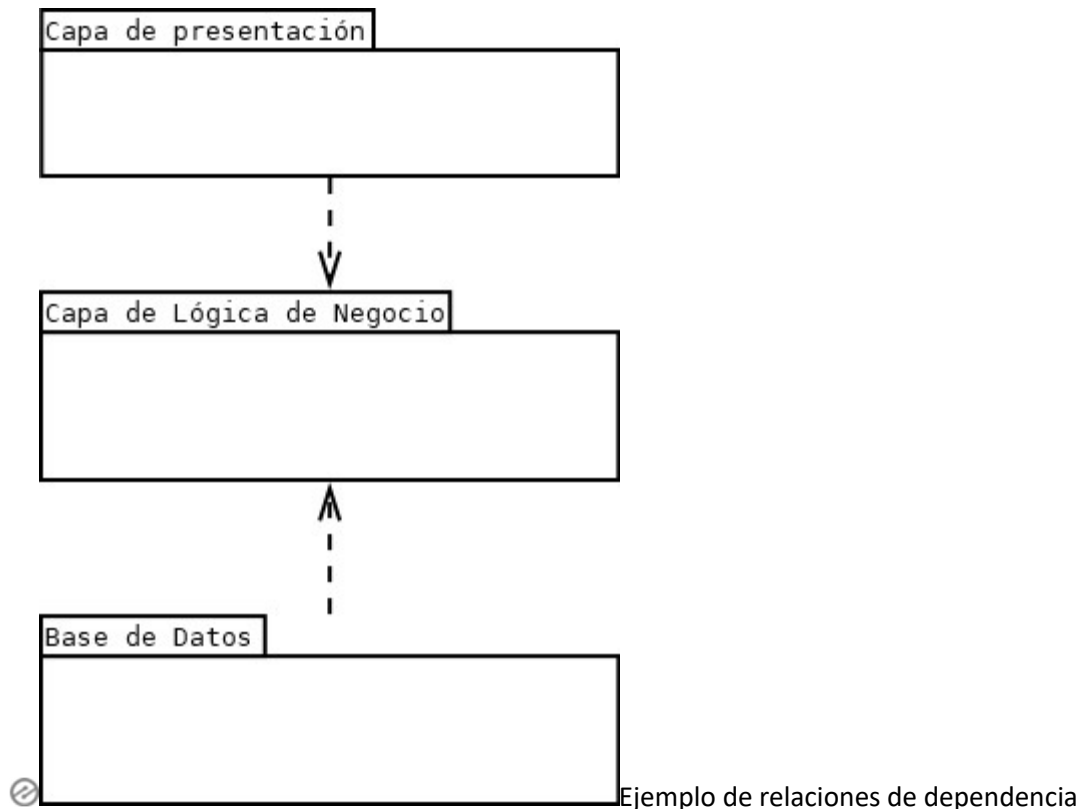
Una dependencia entre paquetes representan que un paquete necesita de los elementos de otro paquete para poder funcionar con normalidad.

Se representa con una flecha discontinua que va desde el paquete que requiere la función hasta el paquete que ofrece esa función.



En esta imagen se dice que el Paquete Origen depende del Paquete Destino para dar su servicio.

Un ejemplo de relación de dependencia que aparece en la realidad un gran número de ocasiones es el siguiente.



Este diagrama, típico de arquitecturas de aplicaciones en tres capas tiene 3 paquetes: Capa de Lógica de Negocio, Capa de Presentación y la Base de datos. Estos dos últimos paquetes (presentación y bbdd) dependen para su funcionamiento de la lógica de negocio.

Ejemplo de un diagrama de paquetes

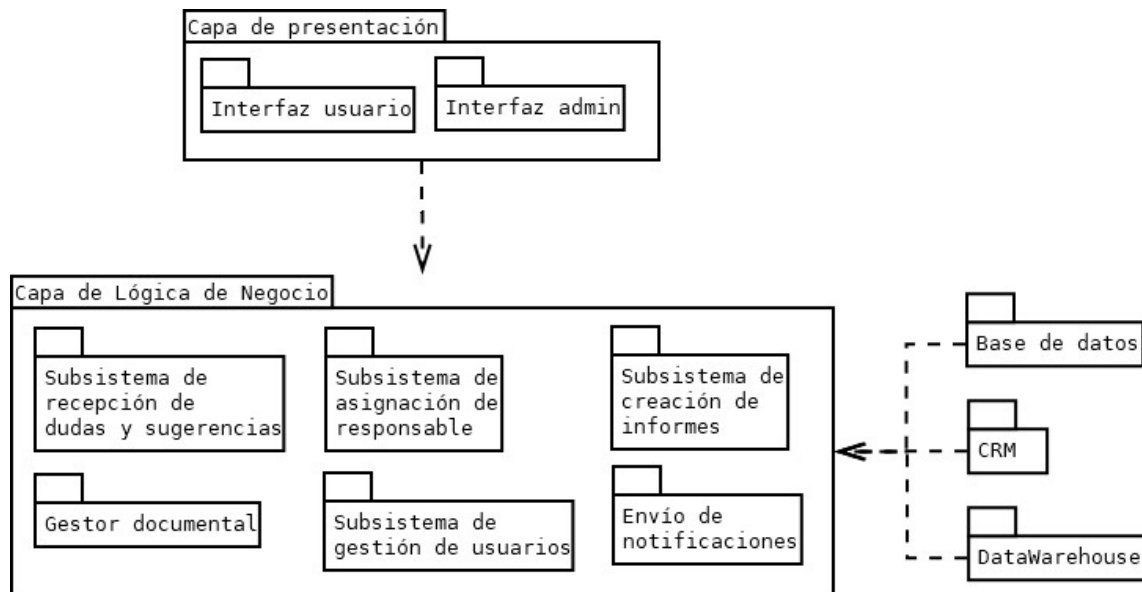
A continuación se muestra, a modo de ejemplo, un diagrama de paquetes de una aplicación:

La aplicación, que tiene como finalidad la recepción y gestión de quejas y sugerencias, estaría compuesta por los siguientes paquetes:



- Capa de presentación. Incluye a su vez los paquetes Interfaz de Usuario e Interfaz Admin
- Capa de Lógica de Negocio, con los siguientes paquetes:
 - Subsistema de recepción de dudas y sugerencias.
 - Subsistema de asignación de responsable.
 - Subsistema de creación de informes.
 - Gestor documental.
 - Subsistema de gestión de usuarios.
 - Envío de notificaciones.
- Base de datos.

- CRM.
- DataWarehouse.



Ejemplo de diagrama de paquetes

Como puedes observar, cada uno de los subpaquetes podría expandirse en otros paquetes, hasta llegar al punto de tener unos paquetes primitivos que no pueden volver a explotarse.

Diagrama de componentes

Mientras que otros diagramas UML describen la funcionalidad de un sistema, los diagramas de componentes se utilizan para **modelar los componentes** que ayudan a hacer esas funcionalidades, representando la forma en la que estos se organizan y sus dependencias.

En esta entrada dedicada al diagramas de componentes veremos qué es un diagrama de componentes, los símbolos de este diagrama y cómo dibujar uno de forma muy sencilla. Al final del artículo podrás encontrar unos cuantos diagramas para ilustrar a modo de ejemplo toda la teoría.

Qué es un diagrama de componentes

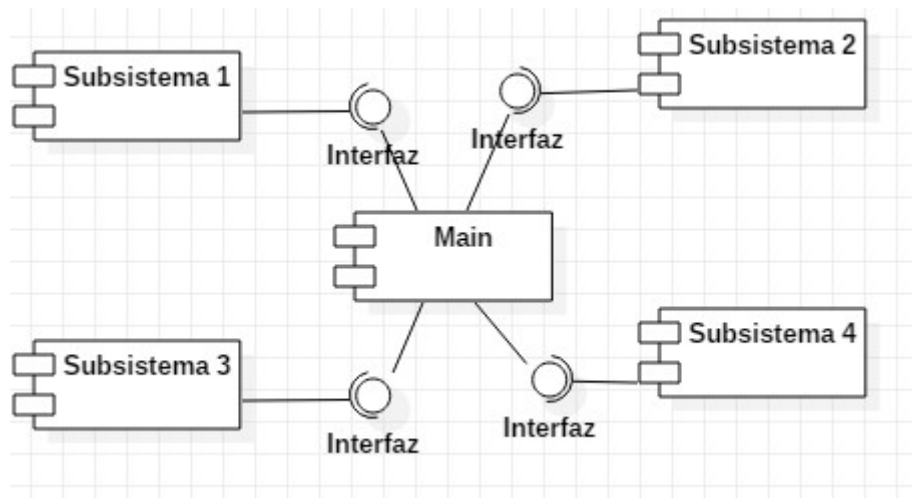
El **diagrama de componentes** es uno de los principales diagramas UML. Está clasificado como diagrama de estructura y, como tal, representa de forma estática el sistema de información. Habitualmente se utiliza después de haber creado el diagrama de clases, pues necesita información de este diagrama como pueden ser las propias clases.

Este diagrama proporciona una vista de alto nivel de los componentes dentro de un sistema. Los componentes pueden ser un componente de *software*, como una base de datos o una interfaz de usuario; o un componente de *hardware* como un circuito, microchip o dispositivo; o una unidad de negocio como un proveedor, nómina o envío.

Algunos usos de este tipo de diagrama es el siguiente:

- Se utilizan en desarrollo basado en componentes para **describir sistemas con arquitectura orientada a servicios**.
- Mostrar la **estructura** del propio código.
- Se puede utilizar para centrarse en la **relación entre los componentes** mientras se ocultan los detalles de las especificaciones.
- Ayudar a **comunicar y explicar** las funciones del sistema que se está construyendo a los interesados o *stakeholders*.

Para su construcción se debe plantear en primer lugar identificar los componentes que utilizará el sistema de información, así como las distintas interfaces. Una forma típica y común para una primera aproximación en sistemas sencillos es utilizar un componente central al que los demás componentes se unen, y que se utiliza como componente gestor del sistema.



Primera

aproximación al diagrama de componentes

Elementos del diagrama de componentes

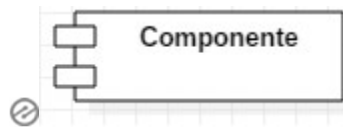
El diagrama de componentes está formado por tres elementos: **Componente, Interfaz y Relación de dependencia**.

Componente

Un componente es un bloque de unidades lógicas del sistema, una **abstracción ligeramente más alta que las clases**. Se representa como un rectángulo con un rectángulo más pequeño en la esquina superior derecha con pestañas o la palabra escrita encima del nombre del componente para ayudar a distinguirlo de una clase.

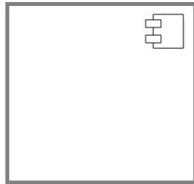
Un componente puede representar dos tipos de elementos: **componentes lógicos** (como por ejemplo componentes de negocio o proceso) o **componentes físicos** (como componentes .NET, EJB...). Por ejemplo, en una aplicación desarrollada en java habrá, con total seguridad, varios componentes «.java», que son componentes lógicos del sistema.

Es representado a través de un rectángulo que tiene, a su vez, dos rectángulos a la izquierda, tal y como se muestra en la siguiente imagen:



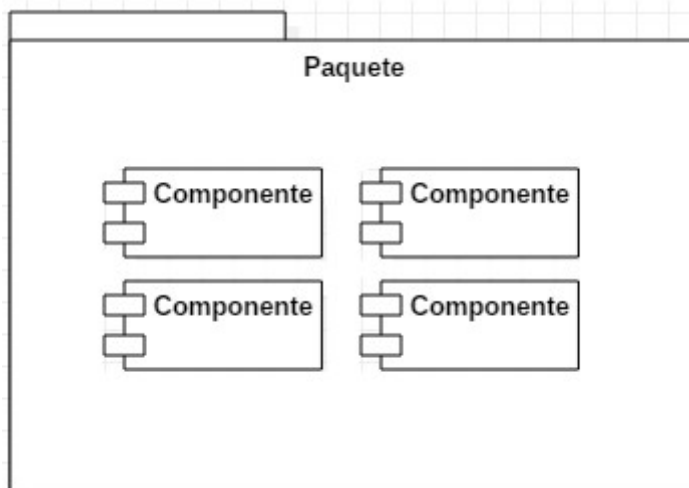
Notación de componente

Otra notación, utilizada en las últimas versiones de UML consiste en un rectángulo con un rectángulo más pequeño en la esquina superior derecha con pestañas.



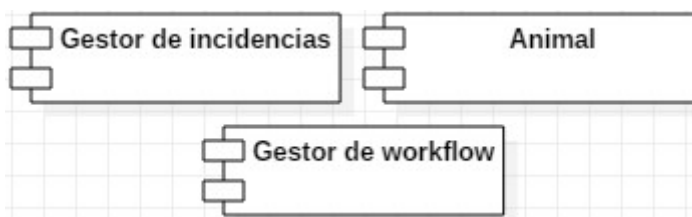
Otra notación de componente

También es posible utilizar el diagrama de paquetes para hacer un conjunto de varios módulos. Con esto se consigue representar la unión de esos módulos para un fin concreto.



Paquete con varios componentes

Ejemplos de componentes podrían ser los siguientes: Gestión de E/S, Animal, Persona, Gestión de incidencias, Gestor de workflow,... Como ves son conceptos muy amplios y que pueden ser más o menos específicos dependiendo de la profundidad que se puede dar al diagrama.



Ejemplos de componentes

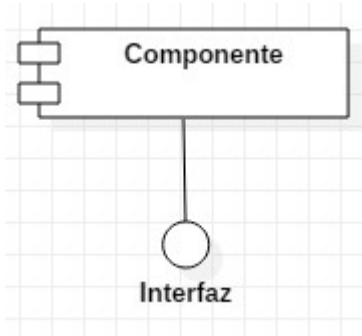
Lo ideal es que los componentes estén diseñados de forma que tengan una gran cohesión y un bajo acoplamiento, para favorecer su reutilización.



Interfaz

La interfaz está siempre asociada a un componente y se utiliza para representar la zona del módulo que es **utilizada para la comunicación** con otro de los componentes.

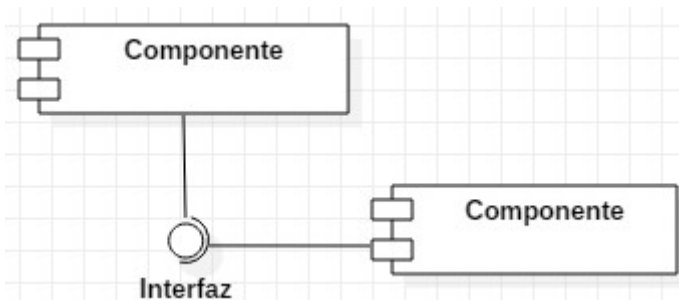
Se representa con una línea que tiene al final un círculo no relleno:



Notación de una interfaz



Otros módulos pueden conectarse a una interfaz. Esto se hace cuando un componente **requiere** o **utiliza** al otro componente mediante su interfaz, que son las operaciones externas que ofrece el componente. Se representa con un línea que termina en un semicírculo que rodea la interfaz del otro componente. En el diagrama se vería de la siguiente manera:



Utilización de interfaz

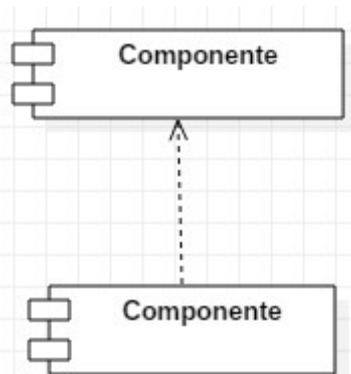
Relación de dependencia

Aunque puedes mostrar más detalles sobre la relación entre dos componentes utilizando la notación de interfaces (interfaz proporcionada y la interfaz requerida), también puedes usar una flecha de dependencia para mostrar la **relación entre dos componentes**. Es una relación más general.



La relación de dependencia representa que un componente requiere de otro para ejecutar su trabajo. Es diferente a la interfaz, pues esta identifica que un componente ofrece una serie de operaciones. En cualquier caso, en ocasiones para simplificar el diagrama no se usan las interfaces sino que solamente se utilizan relaciones de dependencia.

Una relación de dependencia se representa mediante una flecha discontinua que va desde el componente que requiere de otro componente hasta el requerido.



Notación de una relación de dependencia

Las relaciones de dependencia pueden unir, además de componentes con otros componentes, componentes con interfaces.

Cómo dibujar un diagrama de componentes

Puedes utilizar un diagrama de componentes cuando quieras representar tu sistema como una colección de componentes e interfaces. Esto te ayudará a tener una idea de la futura implementación del sistema. Los siguientes son los pasos que pueden servir de guía al dibujar un diagrama de componentes.

- **Paso 1:** Determina el propósito del diagrama e identifica los artefactos como los archivos, documentos, etc. en tu sistema o aplicación que necesitas representar en su diagrama.
- **Paso 2:** A medida que descubres las relaciones entre los elementos que identificaste anteriormente, crea un diseño mental de tu diagrama de componentes.
- **Paso 3:** Al dibujar el diagrama, agrega primero los componentes, agrupándolos dentro de otros componentes como mejor te parezca.
- **Paso 4:** El siguiente paso es agregar otros elementos, como interfaces, clases, objetos, dependencias, etc. al diagrama de componentes y completarlo.
- **Paso 5:** Puede adjuntar notas en diferentes partes de su diagrama de componentes para aclarar ciertos detalles a otros usuarios.

Diagramas de componentes, ejemplos

Estos son algunos ejemplos del diagrama de componentes, cada uno ha sido dibujado a distinto nivel de abstracción.

Diagrama de componentes de una clínica veterinaria.

En este caso se han utilizado paquetes para dar una visión de alto nivel del sistema.

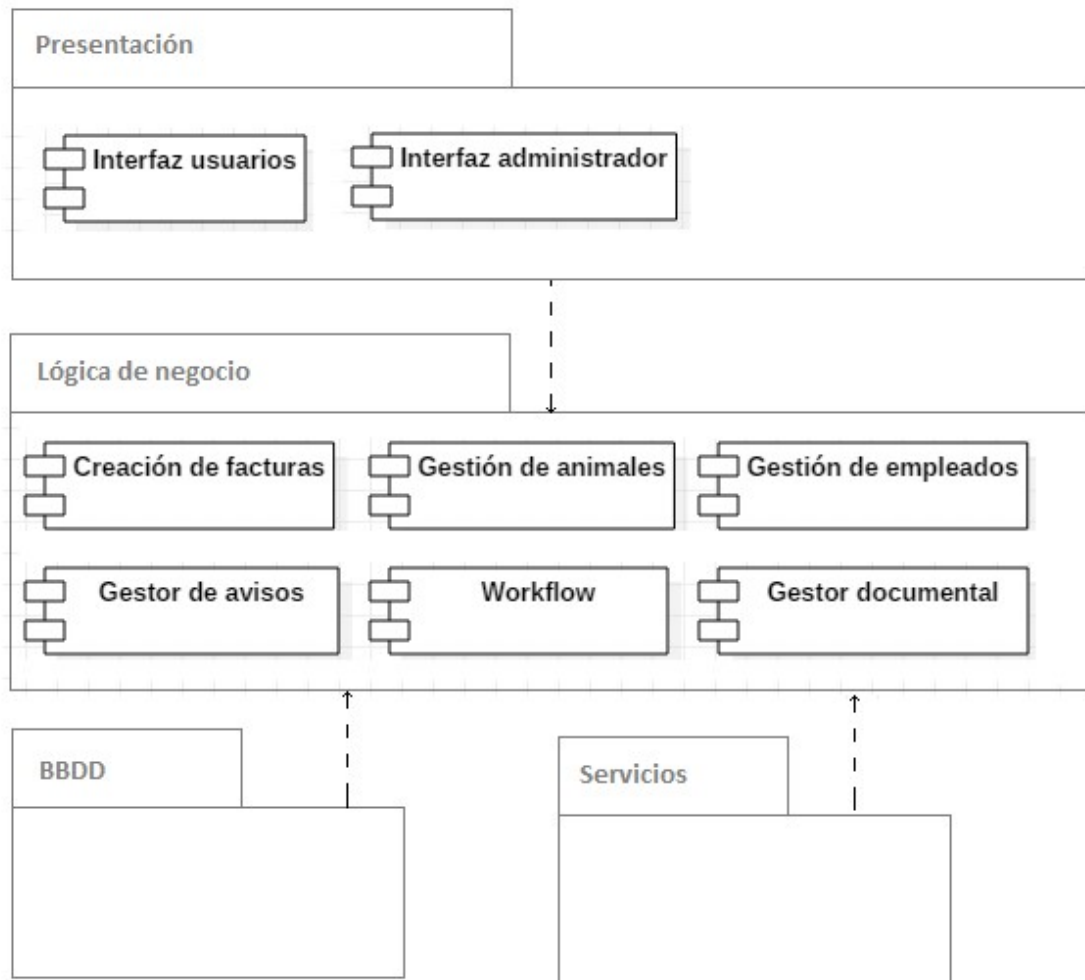
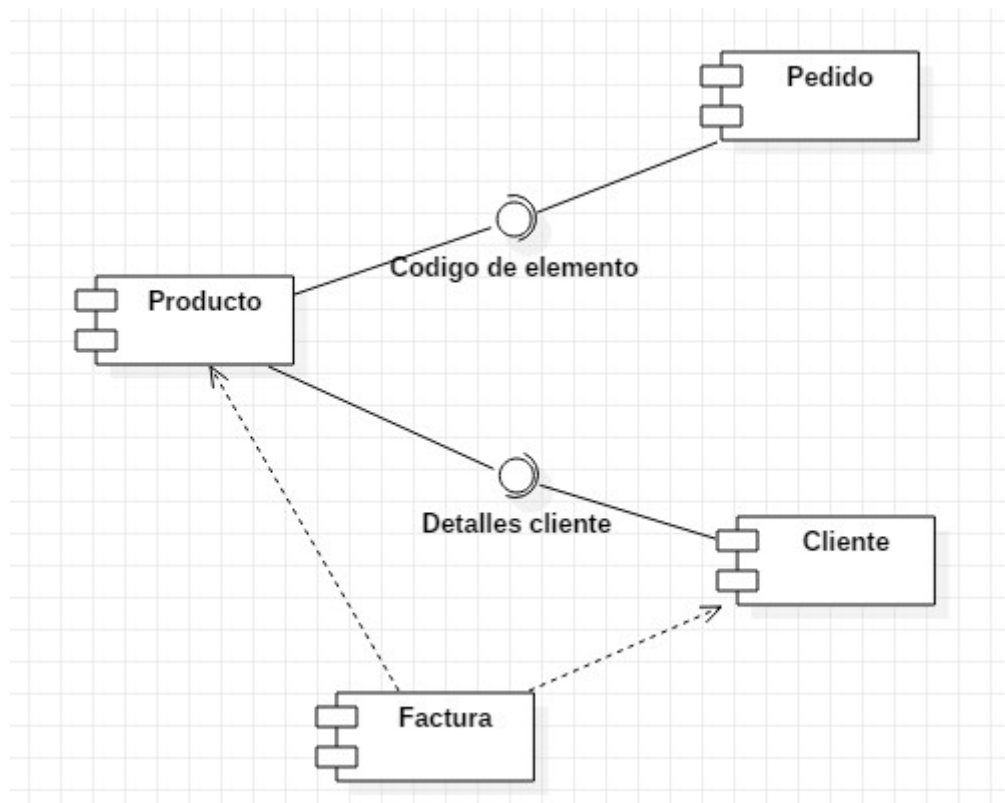


Diagrama de componentes clínica veterinaria

Diagrama de componentes de una tienda online



Diagrama

de componentes tienda online

Diagrama de componentes de un cajero

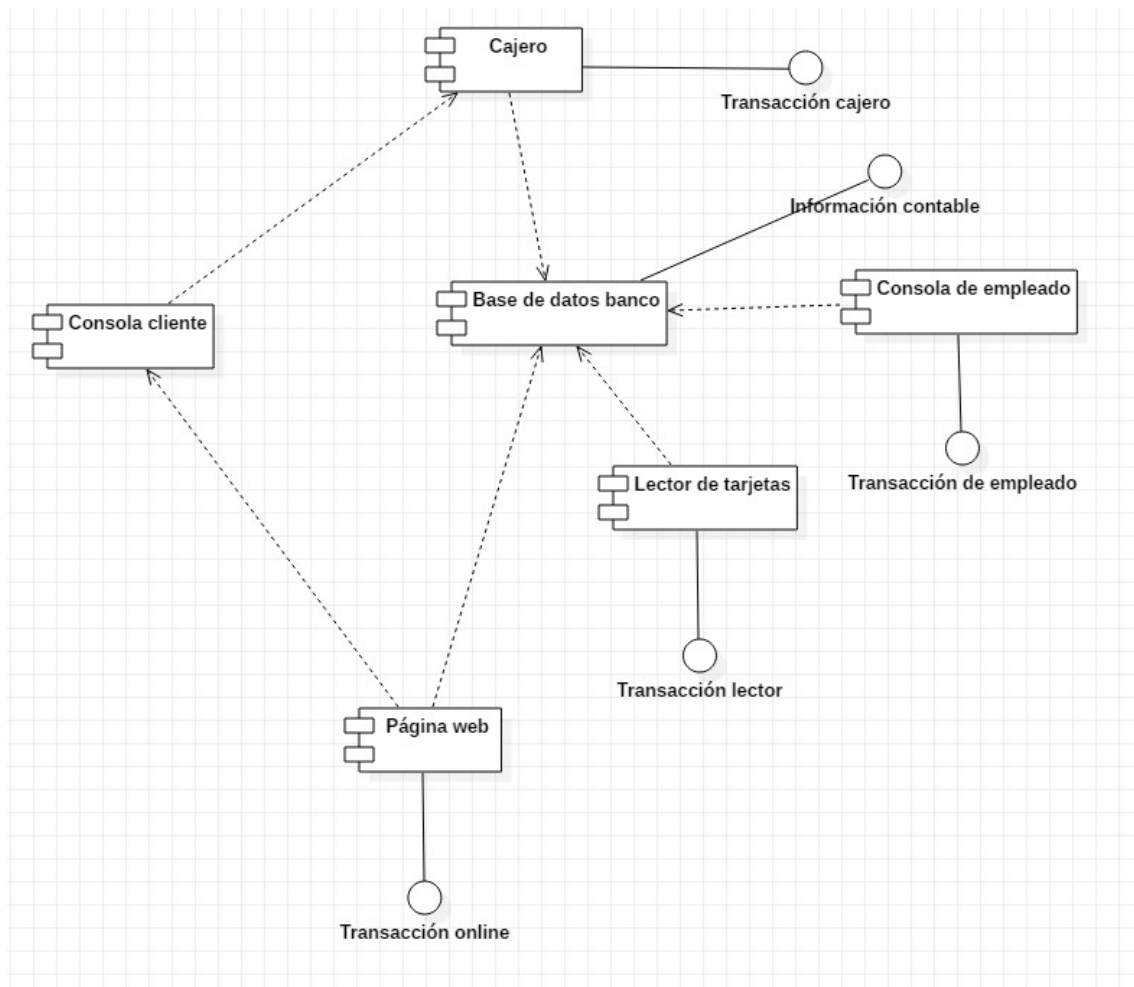


Diagrama de componentes cajero

Diagrama de componentes de gestión de biblioteca

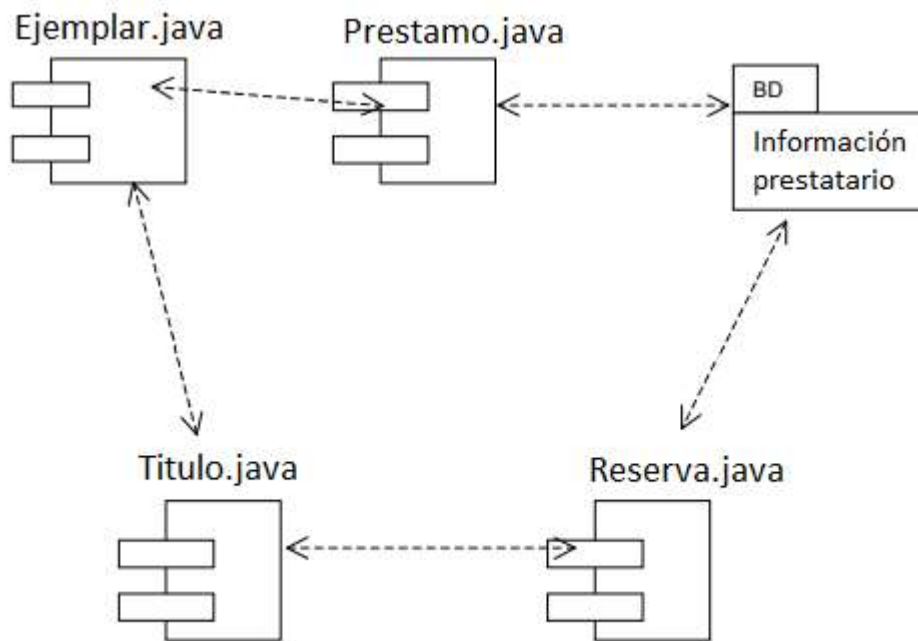


Diagrama de componentes gestión de biblioteca (fuente: Métricav3 «Ministerio de Administraciones Públicas»)

Ejemplo de componentes a incluir en un diagrama de componentes de una tienda online

Aquí te dejo algunos ejemplos de componentes que podrías valorar incluir en un diagrama de componentes de una tienda web:



1. Servidor web: El servidor web es el componente principal que aloja y sirve la página web de la tienda.
2. Base de datos: La base de datos es el componente que almacena la información de los productos, pedidos, clientes y otros datos relevantes de la tienda.
3. Sistema de pagos: El sistema de pagos es el componente encargado de procesar los pagos realizados por los clientes utilizando diferentes métodos de pago, como tarjeta de crédito, transferencia bancaria, entre otros.
4. Sistema de envío: El sistema de envío es el componente que gestiona los envíos de los productos comprados por los clientes, incluyendo la gestión de los proveedores de transporte, el seguimiento del envío y la entrega al cliente final.
5. Servicios web: Los servicios web son componentes que ofrecen una interfaz de programación de aplicaciones (API) para permitir a otros sistemas interactuar con la tienda web, como integraciones con sistemas de pago, sistemas de marketing, redes sociales, entre otros.
6. Gestión de pedidos: El componente de gestión de pedidos es responsable de procesar los pedidos realizados por los clientes, como la validación del pago, la gestión del stock, la preparación del envío y la actualización del estado del pedido.

7. Motor de búsqueda: El motor de búsqueda es el componente que permite a los clientes buscar productos en la tienda, utilizando diferentes criterios de búsqueda, como el nombre, la categoría, el precio, entre otros.
8. Interfaz de usuario: La interfaz de usuario es el componente que permite a los clientes interactuar con la tienda a través de la página web, proporcionando una experiencia de usuario amigable y accesible.
9. Gestión de inventario: El componente de gestión de inventario es responsable de la gestión del stock de los productos en la tienda, incluyendo la actualización de los niveles de stock, la reposición de inventario y la gestión de devoluciones.
10. Seguridad: El componente de seguridad es responsable de garantizar la seguridad de la información y los datos almacenados en la tienda web, como la protección de los datos de los clientes, la prevención de fraudes y la gestión de accesos y permisos.

DIAGRAMAS DE COMPORTAMIENTO

Diagrama de casos de uso

El diagrama de casos de uso es uno de los diagramas incluidos en UML 2.5, estando este clasificado dentro del grupo de **diagramas de comportamiento**. Es, con total seguridad, el diagrama más conocido y es utilizado para representar los actores externos que interactúan con el sistema de información y a través de que funcionalidades (casos de uso o requisitos funcionales) se relacionan. Dicho de otra manera, muestra de manera visual las distintas funciones que puede realizar un usuario (más bien un tipo de usuario) de un Sistema de Información.

En este documento se incluye información sobre como construir este diagrama.

Lo primero es saber cual es su finalidad. El diagrama de casos de uso, dependiendo de la profundidad que le demos, puede ser utilizado para muchos fines, entre ellos podemos encontrar los siguientes:

- **Representar los requisitos funcionales.**
- **Representar los actores** que se comunican con el sistema. Normalmente los actores del sistema son los usuarios y otros sistemas externos que se relacionan con el sistema. En el caso de los usuarios hay que entender el actor como un “perfil”, pudiendo existir varios usuarios que actúan como el mismo actor.
- **Representar las relaciones** entre requisitos funcionales y actores.
- **Guiar el desarrollo** del sistema. Crear un punto de partida sobre el que empezar a desarrollar el sistema.
- **Comunicarse de forma precisa entre cliente y desarrollador.** Simplifica la forma en que todos los participantes del desarrollo, incluyendo el cliente, perciben como el sistema funcionará y ofrecerá una visión general común del mismo.

Elementos de un diagrama de casos de uso

Un diagrama de casos de uso está compuesto, principalmente, de 3 elementos: **Actores, Casos de uso y Relaciones**.

Actores

Como ya hemos comentado en la presentación, un actor es algo o alguien externo al sistema que interactúa de forma directa con el sistema. Cuando decimos que interactúa nos referimos a que aporta información, recibe información, inicia una acción...

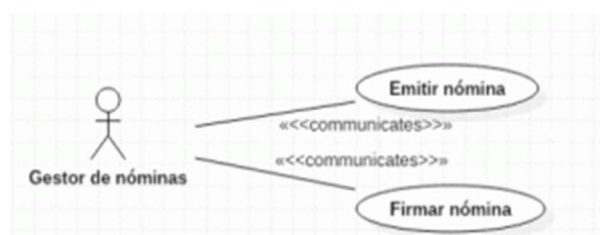
Se representan con una imagen de un “muñeco de palo” con el nombre del actor debajo.



Representación de un actor

Existen dos tipos de actores: Los usuarios y los sistemas.

No hay que entender los usuarios como personas singulares, sino como “perfiles o roles” que identifican a un tipo de usuario, pero no al usuario en sí. Por ejemplo, en una aplicación de gestión de nóminas, un actor de este tipo podría ser “gestor de nóminas” que se encarga de emitir y firmar nóminas. Este rol podría ser tomado, por ejemplo, por cualquier individuo del personal de recursos humanos y, además, por el jefe de la empresa. Es un ejemplo muy sencillo, pero como puedes ver, un actor no representa a una única persona o a un único usuario.



Ejemplo de actor

Por otro lado, los actores pueden ser otros sistemas que también interactúan con nuestro propio sistema. Un ejemplo podría ser, en nuestra aplicación de nóminas, un sistema que almacene las nóminas firmadas a modo de archivo. En este caso cuando se firma la nómina se recibe la misma por el sistema de archivo, por tanto el caso de uso se relaciona con el actor.

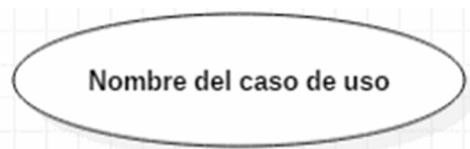
En ocasiones este tipo de actores no se representa con un “hombre de palo” porque puede dar la sensación de que es un usuario y queda poco intuitivo.

Casos de uso

Un caso de uso se utiliza para representar una de las funcionalidades que realiza el sistema. Es una secuencia de acciones que hace el sistema y que producen un resultado que puede percibir un usuario.

Formalmente hablando, un caso de uso es una clasificación de comportamiento que especifica una unidad de funcionalidad completa y que está realizada por uno o más sujetos que se relacionan con el caso de uso colaborando para ello con uno o más actores y que produce un resultado que tiene alguna utilidad para cualquier de esos actores.

Se representan con una elipse que incluye en su interior el nombre del caso de uso.



Representación de un caso de uso

Existen muchos ejemplos de casos de uso. Algunos podrían ser: Crear pedido, Listar productos, Enviar correo. Cualquier acción que realice la aplicación.



Las especificaciones anteriores a UML 2.5 requerían que un caso de uso sea invocado por un actor. En UML 2.5 esto se eliminó, lo que significa que podría haber algunas situaciones en las que la funcionalidad del sistema la inicie el propio sistema y, al mismo tiempo, brinde resultados útiles a un actor. Por ejemplo, el sistema podría notificar a un cliente que se envió la orden, programar la limpieza y el archivo de la información del usuario, solicitar información de otro sistema, etc.

Relaciones

Las relaciones **conectan los casos de uso** con los actores o los casos de uso entre sí.

Cuando conectan un actor con un caso de uso representa que ese actor **interactúa** de alguna manera con ese caso de uso y se representa con una línea continua con la identificación `<<communicates>>`.



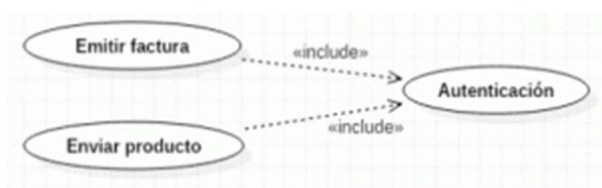
Cuando conectan casos de uso entre sí se pueden diferenciar dos tipos de relaciones: `<<include>>` y `<<extends>>`. En español a veces se usa la nomenclatura `<<usa>>` y `<<extiende>>`:

- `<<include>>`: Se utiliza para representar que un caso de uso **utiliza siempre** a otro caso de uso. Es decir, un caso de uso se ejecutará obligatoriamente (lo incluye, lo usa). Se representa con una flecha discontinua que va desde el caso de uso de origen al caso de uso que se incluye.



Relación include entre dos casos de uso

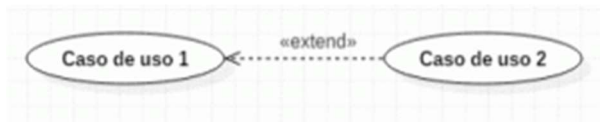
Un uso típico de este tipo de relaciones se produce cuando dos casos de uso **comparten una funcionalidad**. Esa funcionalidad es extraída de los dos y se crea un caso de uso nuevo que se relaciona con los anteriores con un include.



Ejemplo de uso de include

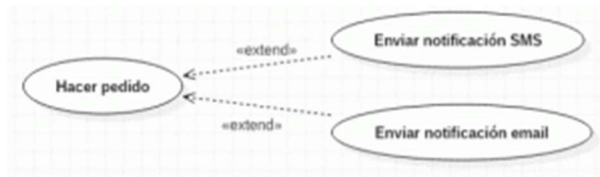
En este ejemplo, los casos de uso emitir factura y enviar producto ejecutarán ambos el caso de uso autentificación.

- **<<extend>>**: Este tipo de relaciones se utilizan cuando un caso de uso tiene un comportamiento **opcional**, reflejado en otro caso de uso. Es decir, un caso de uso puede ejecutar, normalmente dependiendo de alguna condición o flujo del programa, otro caso de uso. Se representa con una flecha discontinua que va desde el caso de uso opcional al original.



Relación extend entre dos casos de uso

Un ejemplo de esta relación podría ser la siguiente:



Ejemplo de relaciones extend

En este supuesto el caso de uso Hacer pedido puede dar lugar (o no) a otros dos casos de uso: Enviar notificación SMS y Enviar notificación email. Se supone que, cuando un usuario hace un pedido, el sistema le permite elegir si quiere que se envíe una notificación de ese pedido por SMS o por email.



Existe, además, otra relación denominada **generalización** que consiste en hacer que un elemento herede el comportamiento de otro. Aunque se puede utilizar entre casos de uso, es más común utilizarlo entre actores, haciendo que uno de los actores tenga acceso a las funcionalidades de otro. Se representa con una flecha con la punta hueca que va desde el elemento que hereda al elemento heredado:



Generalización entre dos actores

Si estás interesado, puedes seguir nuestra [guía para identificar actores y casos de uso](#).

Descripción de requisitos funcionales y no funcionales

Es común en este tipo de diagramas describir cada caso de uso junto con la secuencia de pasos necesaria para completarlo y las posibles excepciones hasta definir todas las situaciones posibles. Esta descripción servirá de guía para el desarrollo, la profundidad de las situaciones que se traten dependerá de cada fase del proyecto o de cada situación en particular.

Existen dos tipos de requisitos:



- Requisitos funcionales
- Requisitos no funcionales

Los requisitos suelen ser plasmados junto a la siguiente información:

- **Identificador y nombre descriptivo:** Se utiliza una identificación única para diferenciarlo de los demás y un nombre descriptivo que suele coincidir con el objetivo que los actores esperan alcanzar al realizar el caso de uso.
- **Versión**
- **Autores**
- **Objetivos asociados**
- **Requisitos asociados**
- **Descripción:** Este campo debe completarse de forma distinta en función de si el caso de uso es abstracto o concreto.
- **Precondición:** se expresan en lenguaje natural las condiciones necesarias para que se pueda realizar el caso de uso.
- **Secuencia normal:** secuencia normal de interacciones del caso de uso. En cada paso, un actor o el sistema realiza una o más acciones, o se realiza otro caso de uso.
- **Postcondición:** se expresan en lenguaje natural las condiciones que se deben cumplir después de la terminación normal del caso de uso.
- **Excepciones:** especifica el comportamiento del sistema en el caso de que se produzca alguna situación excepcional durante la realización de un paso determinado, lo que modifica el flujo «normal» del caso de uso.
- **Importancia**
- **Urgencia**
- **Comentarios**

Ejemplos de representación de un requisito en forma de tabla:

RF-01	Acceso Aplicación	
Versión	Versión 1.0	
Autores	Alonso Quijano	
Objetivos Asociados	OBJ-01: Acceso Controlado a la Aplicación Software.	
Requisitos asociados	RI-01: Información de los Usuarios.	
Descripción	El sistema deberá comportarse como se describe en el siguiente caso de uso cuando un usuario decida acceder a la aplicación.	
Precondición	El usuario tiene que disponer de un nombre de usuario y una contraseña para poder acceder y deberá tener el acceso habilitado.	
Secuencia normal	Paso	Acción
	1	El usuario solicita al sistema entrar en la aplicación.
	2	El sistema solicita al usuario que introduzca el nombre de usuario y su contraseña.
	3	El usuario introduce su nombre y su contraseña.
	4	El sistema comprueba los datos introducidos.
	5	Si los datos son correctos el sistema muestra la página de inicio de la aplicación.
Excepciones	Paso	Acción
	5	Si el nombre de usuario no es correcto. El sistema muestra un mensaje. Ir al paso 2.
	5	Si la contraseña no es correcta. El sistema muestra un mensaje. Ir al paso 2.
	5	Si el usuario no tiene el acceso habilitado a la aplicación. El sistema muestra un mensaje. Ir al paso 2.
Postcondición	Si el nombre de usuario y la contraseña son correctos accede a la pantalla de inicio de la aplicación.	
Importancia	Vital.	
Urgencia	Inmediatamente.	
Comentarios	Ninguno.	

Modelado de un requisito funcional

RNF-01	Entorno de Explotación
Versión	Versión 1.0
Autores	Alonso Quijano
Objetivos asociados	OBJ-05: Funcionamiento óptimo por usuario estándar
Requisitos asociados	
Descripción	El sistema deberá funcionar sin ningún tipo de limitación en equipos con: Pentium IV a 2,4 GHz, con 1 <u>GBytes</u> de RAM y al menos 6 <u>GBytes</u> de disco duro.
Importancia	Vital.
Urgencia	Inmediatamente.
Estabilidad	Alta.
Comentarios	Ninguno.

Modelado de un requisito no funcional

Cómo dibujar un diagrama de casos de uso

A la hora de dibujar un diagrama de casos de uso te recomendamos que compruebes que has realizado previamente todas estas tareas, respondiendo a las preguntas que te escribimos a continuación:

- **Recopilar fuentes de información:** ¿cómo se supone que debo saber eso?
- **Identificar actores potenciales:** ¿qué usuarios utilizan los bienes y servicios del sistema empresarial?. Para más información te recomiendo la [guía para identificar actores y casos de uso](#).
- **Identificar posibles casos de uso:** ¿a qué bienes y servicios pueden recurrir los actores?
- **Conectar** los casos de uso: ¿quién puede hacer uso de los bienes y servicios del sistema empresarial?
- **Describir actores:** ¿a quién o qué representan los actores?
- **Buscar más casos de uso:** ¿Qué más debe hacer el sistema?
- **Documentar** casos de uso: ¿qué sucede exactamente en cada caso de uso?
- **Relacionar modelos** entre casos de uso empresarial: ¿qué actividades se realizan repetidamente?
- Verificar la vista, **¿todo es correcto?**

Los pasos se han escrito en este orden a propósito, ya que es la forma lógica de seguirlos. Sin embargo, este orden no es obligatorio, ya que en la práctica, los pasos individuales a menudo se superponen unos con otros.



Para poder seguir los pasos de una forma óptima, es importante comprender el negocio/sistema para conseguir seguir cada paso individual. En algunos casos también es necesario consultar a

los expertos o consultores del negocio. No tiene sentido aferrarse a la visión personal del analista, si este no tiene mucho conocimiento del área de negocio de la aplicación.

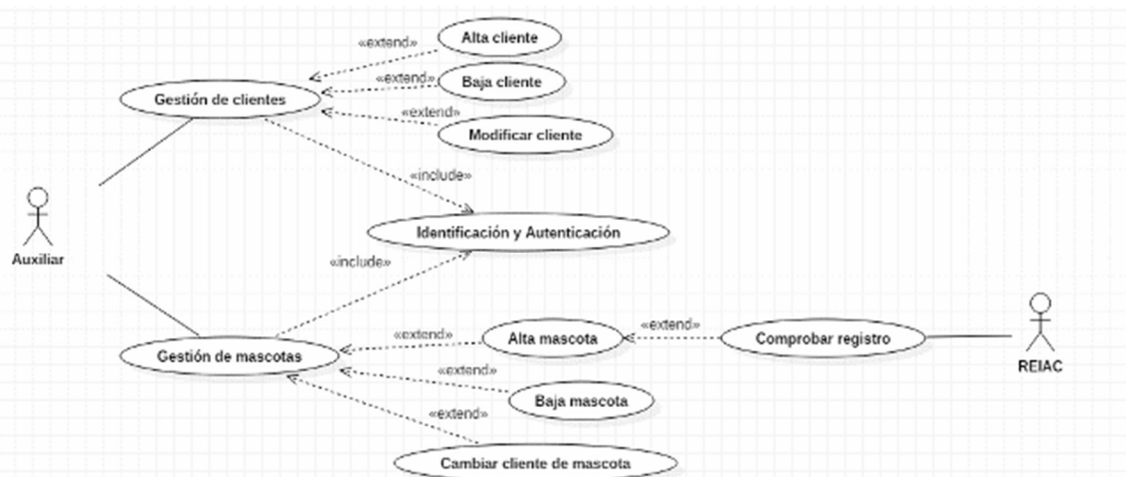
Ejemplos de un diagrama de casos de uso

Ejemplo clínica veterinaria:
A modo de ejemplo se propone un ejercicio de un diagrama de casos de uso que consiste en el diseño de una aplicación que gestione los tramites a realizar en una clínica veterinaria en base a las siguientes premisas:

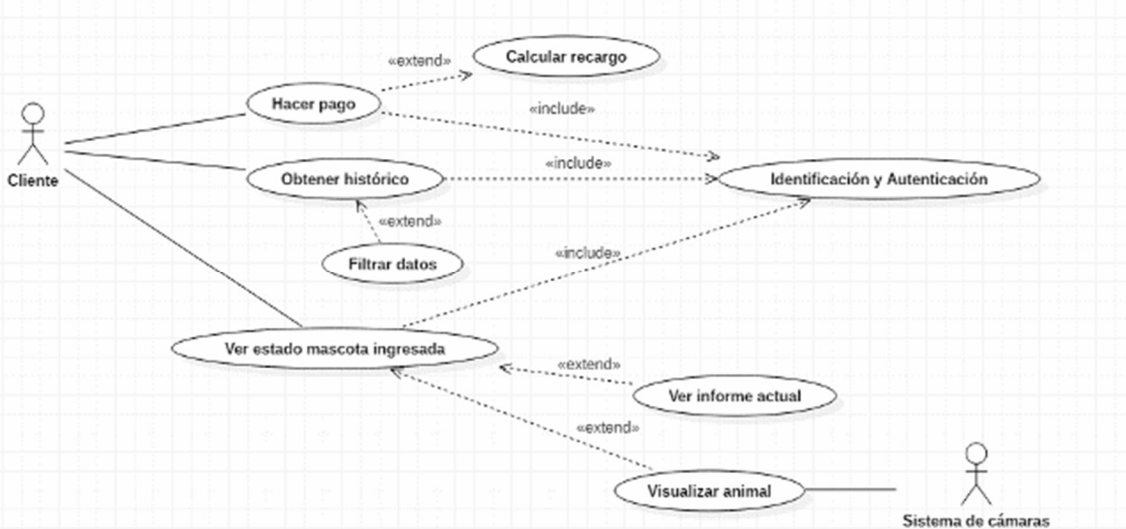
- La clínica veterinaria almacena datos de contacto de todos sus clientes como pueden ser: Nombre, Apellidos, DNI, Fecha de nacimiento, Teléfono o Email. Estos datos son introducidos y gestionados por los auxiliares, que ejercen las funciones administrativas.
- Además se almacena información de cada uno de las mascotas de las que es dueño cada cliente. Obviamente, cada cliente puede tener más de una mascota, pero cada mascota solo puede pertenecer a un único cliente. Se permite, además, cambiar el dueño de una mascota por otro.



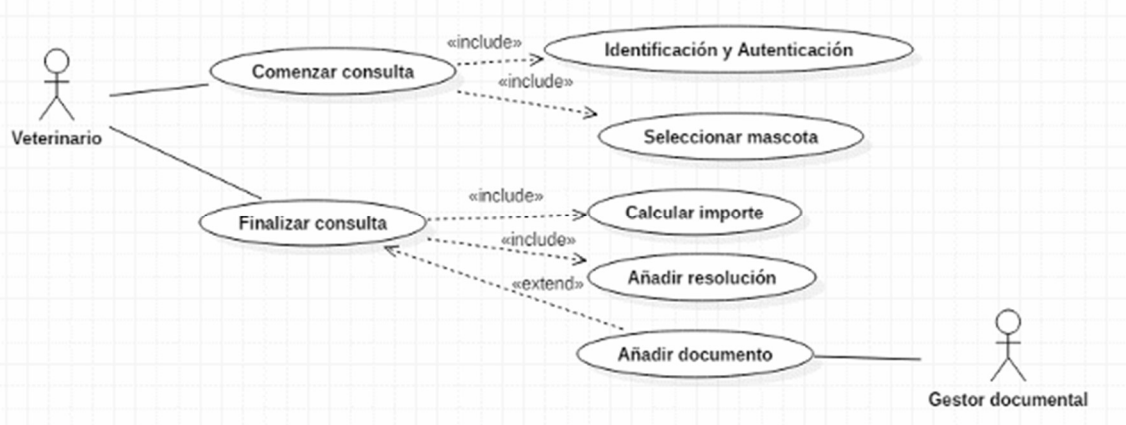
- Al dar de alta un nuevo animal, se comprobará en el registro del REIAC (Red Española de Identificación de Animales de Compañía) si el animal está correctamente dado de alta. Este proceso unicamente se hará en animales que tengan la obligación de estar identificados.
- Cada vez que un veterinario realiza una consulta sobre un animal, esta queda almacenada incluyendo datos básicos como: Tiempo de consulta, Identificación de la persona que lo ha tratado, Animal tratado, Importe total, Resolución, Recetas... Para calcular el tiempo de la consulta el veterinario tendrá un botón en la aplicación donde pueda pulsar cuando comienza la consulta para calcular el tiempo a modo de cronómetro y otro botón para finalizar.
- En caso de que el animal se quede ingresado en la clínica, el cliente debe ser capaz de acceder al estado en tiempo real del animal. Además podrá comunicarse con una cámara que tendrá el animal colocada, donde podrá ver su situación actual. La gestión de estas cámaras no corresponde al sistema, sino que se utilizará una aplicación ya presente en el veterinario.
- Las recetas y otros documentos relacionados con el servicio se incluirán en un gestor de contenidos que ya está en funcionamiento en la clínica veterinaria.
- Una vez terminado el servicio, el cliente no tiene por qué realizar inmediatamente el pago, sino que puede identificarse posteriormente en la aplicación vía web y realizar el pago. Si el cliente tarda más de una semana se efectuará un recargo sobre el precio inicial.
- Además, el cliente debe ser capaz de obtener un histórico de todas las consultas que ha recibido cualquiera de sus mascotas.



Ejemplo Diagrama de casos de uso del actor «auxiliar»



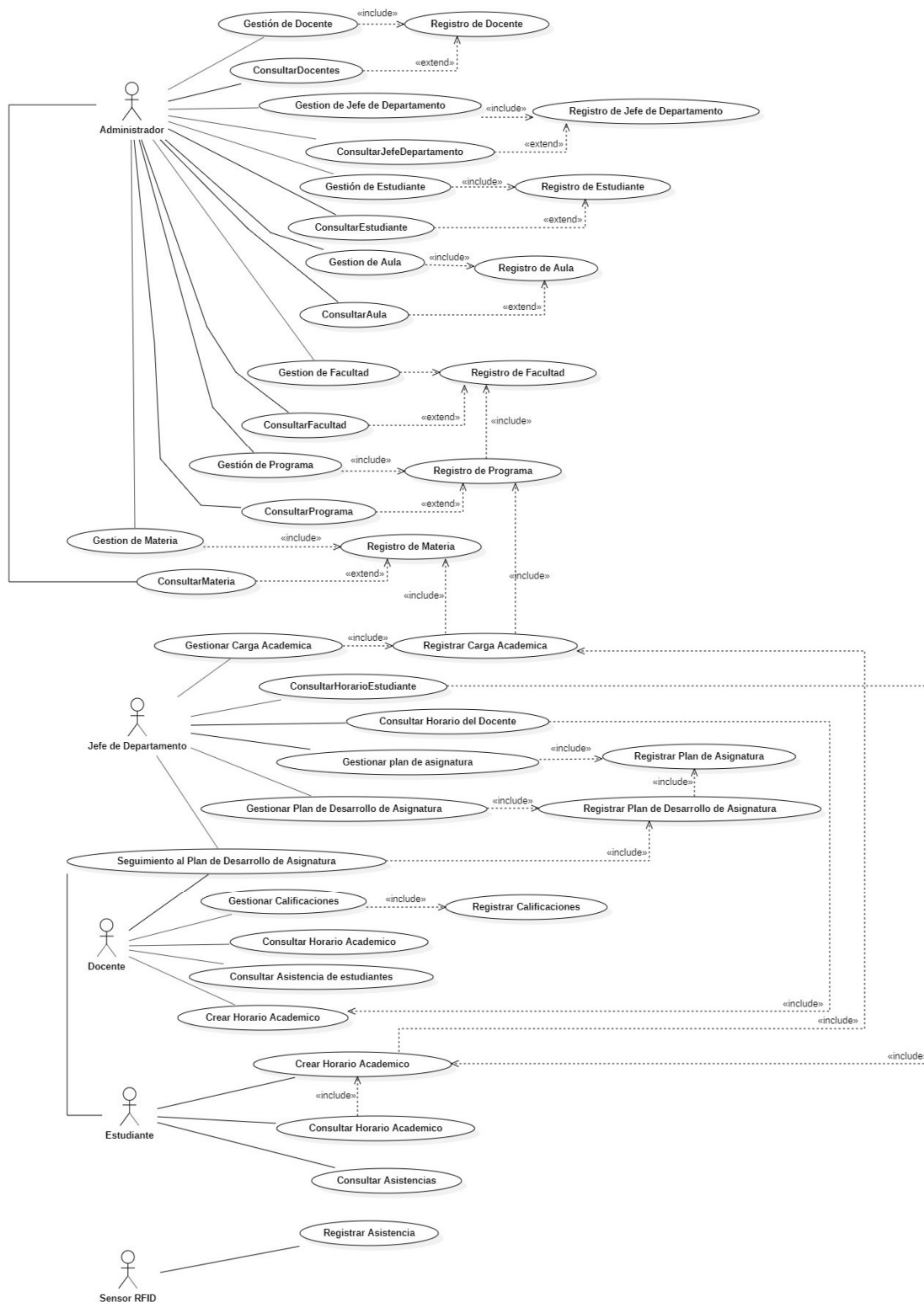
Ejemplo Diagrama de casos de uso del actor «Cliente»



Ejemplo Diagrama de casos de uso del actor «veterinario»

No obstante, dependiendo del nivel de profundidad, el diagrama puede variar significativamente descomponiendo, añadiendo, omitiendo o fusionando alguno de los casos de uso que se han expuesto.

Ejemplo centro educativo:



Ejemplo diagrama casos de uso centro educativo

Ejemplos de casos de uso para un diagrama de casos de uso de una página web

Aquí te dejo un ejemplo de casos de uso que podrías incluir en un diagrama de casos de uso de una tienda online, que pueden servir de punto de partida para plantear el diagrama:

1. Registrarse en la tienda: El usuario puede crear una cuenta en la tienda para poder comprar productos.
2. Iniciar sesión: El usuario puede iniciar sesión en la tienda para acceder a su cuenta y comprar productos.
3. Buscar productos: El usuario puede buscar productos en la tienda por nombre, categoría o palabra clave.
4. Ver detalles de un producto: El usuario puede ver detalles de un producto, incluyendo su descripción, precio, stock y otros detalles relevantes.
5. Añadir producto al carrito: El usuario puede añadir un producto al carrito de compras para comprarlo más adelante.
6. Ver carrito de compras: El usuario puede ver el contenido de su carrito de compras y realizar cambios antes de finalizar la compra.
7. Realizar compra: El usuario puede finalizar la compra de los productos en su carrito de compras.
8. Ver historial de compras: El usuario puede ver su historial de compras anteriores.
9. Ver estado del pedido: El usuario puede ver el estado de su pedido actual, incluyendo el tiempo de envío estimado y la fecha de entrega.
10. Realizar pagos: El usuario puede realizar pagos utilizando los diferentes métodos de pago disponibles en la tienda.
11. Gestionar cuenta: El usuario puede gestionar su cuenta de usuario, actualizar sus datos personales, cambiar su contraseña y cerrar sesión.
12. Administrar productos: El administrador de la tienda puede agregar, eliminar o actualizar los productos disponibles en la tienda.
13. Administrar pedidos: El administrador de la tienda puede ver y actualizar el estado de los pedidos de los clientes.
14. Administrar usuarios: El administrador de la tienda puede ver y actualizar los datos de los usuarios registrados en la tienda.

Diagrama de secuencia

El diagrama de secuencia es un tipo de diagrama de interacción contenido en UML 2.5. Su objetivo es representar el intercambio de mensajes entre los distintos objetos del sistema para cumplir con una funcionalidad. Define, por tanto, el comportamiento dinámico del sistema de información.

Normalmente es utilizado para definir como se realiza un caso de uso por lo que es comúnmente utilizado junto al [diagrama de casos de uso](#). También se suele construir para comprender mejor el [diagrama de clases](#), ya que el diagrama de secuencia muestra como objetos de esas clases interactúan haciendo intercambio de mensajes.

Construcción del diagrama de secuencia

El diagrama de secuencia está construido a partir de dos dimensiones:

- Horizontal: Representa los objetos que participan en la secuencia.
- Vertical: Representa la línea de tiempo sobre la que los elementos actúan. Va de arriba (menor tiempo) hacia abajo (mayor tiempo) No es común «reglar» esta dimensión mediante una escala para poner tiempos específicos, a excepción de sistemas de tiempo real donde la velocidad para llevar a cabo la funcionalidad sí es relevante.



diagrama de secuencia

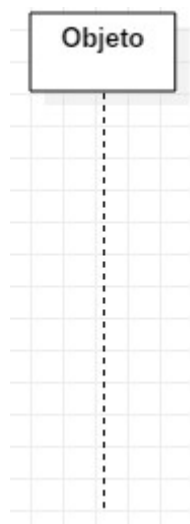
El diagrama de secuencia está compuesto por dos elementos: Objetos y mensajes.

Objeto

Un objeto representa a un participante en la interacción. Un objeto puede ser una instancia de una clase, un módulo, un grupo de clases, ... en definitiva, un objeto es un componente software que tiene una funcionalidad específica. Dependerá del nivel de abstracción la representación de cada objeto.

Es importante destacar que, en el diagrama de secuencia y a diferencia de otros diagramas, cada uno de los objetos añadidos al diagrama representa solamente una instancia de ese objeto y no varias. En caso de tratarse de un elemento multivaluado, se debe dejar constancia de cual es el elemento que está trabajando.

Un objeto se representa mediante un rectángulo que incluye un identificador en su interior y del que sale una línea de forma vertical hacia abajo. Esta línea se llama línea de vida y representa el tiempo en el que un objeto está presente.

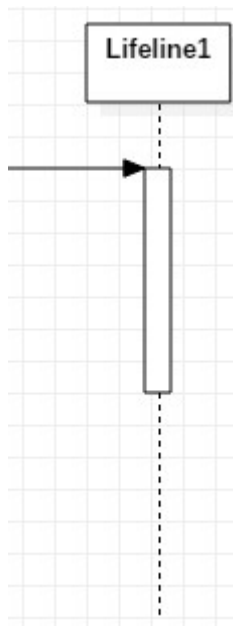


Notación de un objeto

Los objetos que existen previamente al comienzo de la interacción se sitúan en el eje horizontal mientras que los objetos que se crean durante el transcurso de la misma se sitúan en el momento de la creación.

El final de un punto de vida se representa con un aspa al final de la línea de vida, aunque puede no ser eliminado nunca, prologando su línea de vida hasta el final del diagrama.

Los objetos contienen el denominado foco de control que no es más que el tiempo en el que tal objeto está llevando a cabo algún trabajo. Se representa mediante un rectángulo superpuesto a la línea de vida.

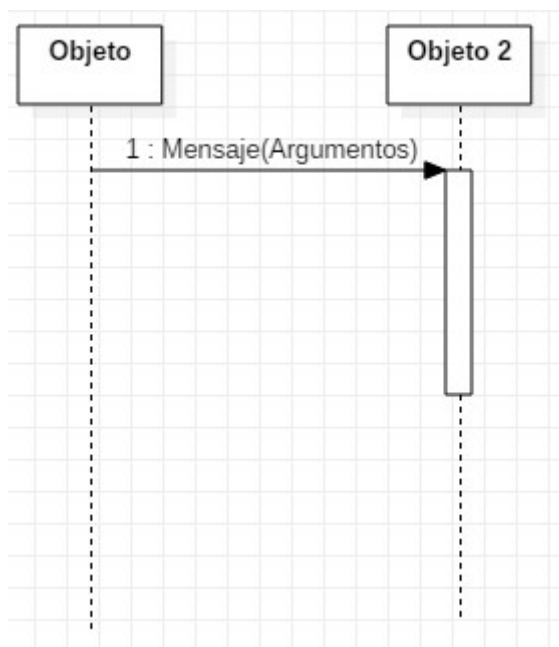


Notación foco de control

Mensaje

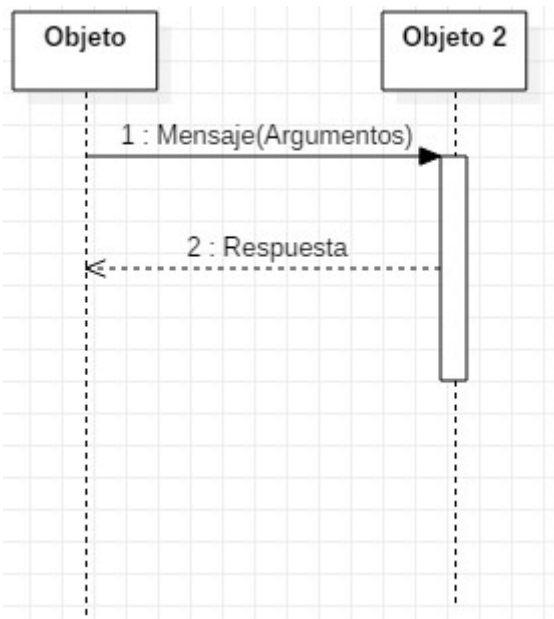
Se utiliza un mensaje en el diagrama de secuencia para representar el paso de un mensaje entre dos objetos o entre un objeto y sí mismo.

Se representa utilizando una flecha que incluye el nombre del mensaje y los argumentos que incluye y que va desde el objeto que envía el mensaje hasta el objeto que lo recibe.



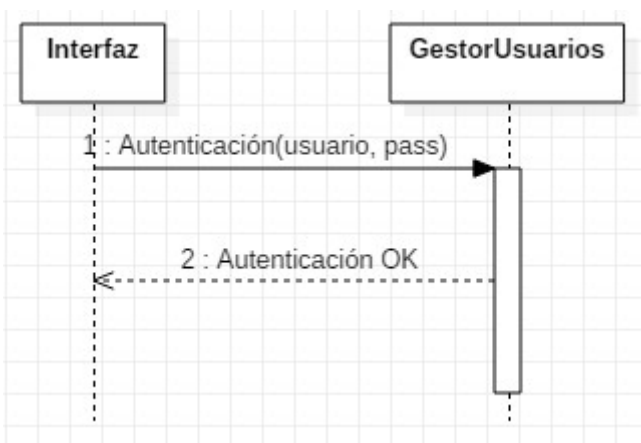
Notación de un mensaje

A veces el objeto que recibe el mensaje envía una respuesta. Esta respuesta se representa con una flecha discontinua.



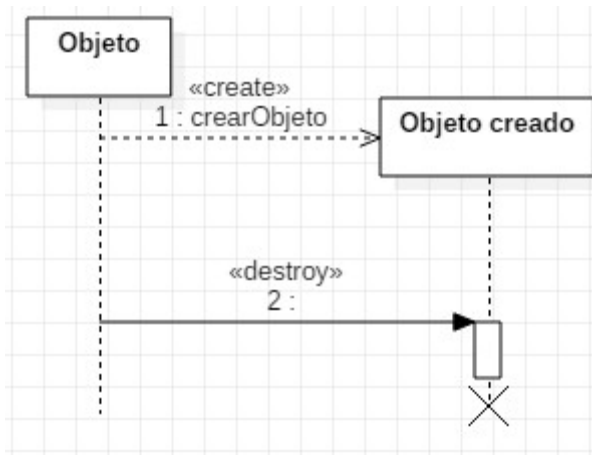
Notación de una respuesta

Un ejemplo de mensaje podría ser el siguiente:



Ejemplo de mensajes

Como hemos visto antes, un objeto puede ser creado a mitad de la interacción. Esta creación se hace a través de otro objeto mediante una llamada create. También existe la posibilidad de que un objeto destruya otro. Ambas acciones se representan de la siguiente manera:

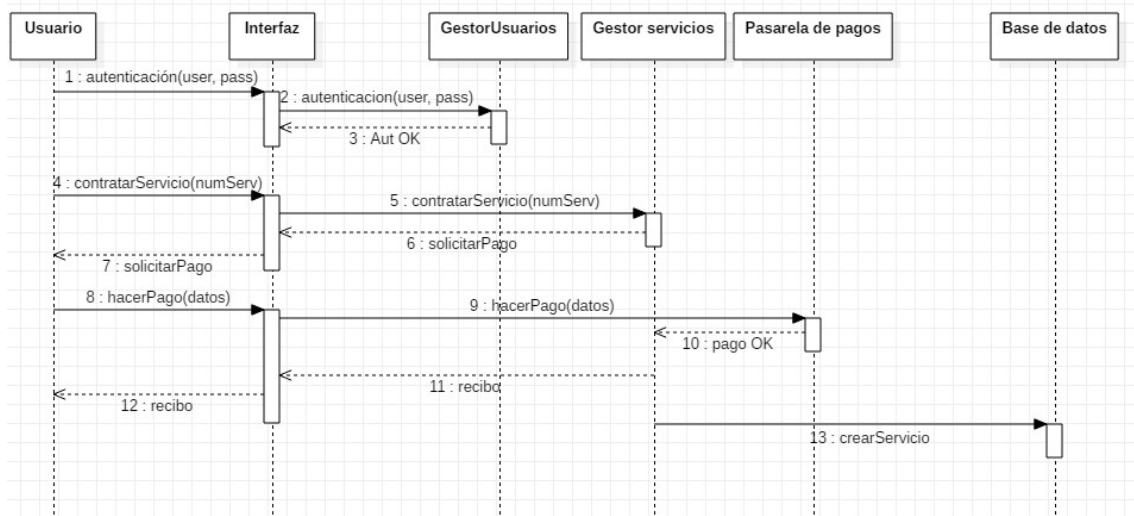


Notación create y destroy



Ejemplo de diagrama de secuencia

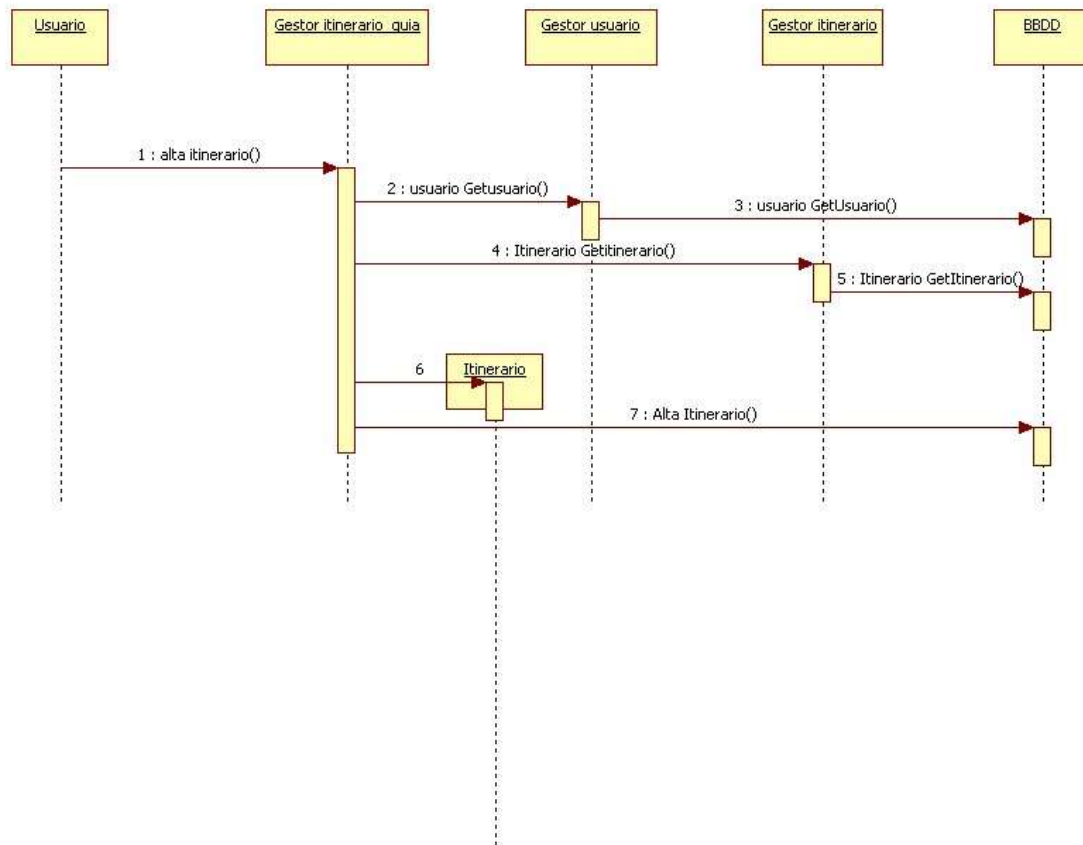
A continuación se muestra un ejemplo de un diagrama de secuencia de la funcionalidad contratar servicio.



Ejemplo de diagrama de secuencia



Este es otro ejemplo, para un caso de uso gestionar itinerario:



Ejemplo 2 diagrama de secuencia

Diagrama de actividades

Un diagrama de actividades en UML (Lenguaje de Modelado Unificado) es un tipo de diagrama que se utiliza para modelar el flujo de trabajo o el comportamiento de un sistema o proceso de negocio. Es útil para representar procesos, procedimientos y algoritmos complejos en una forma visual fácil de entender.

Los diagramas de actividades muestran una secuencia de acciones, un flujo de trabajo que va desde un punto inicial hasta un punto final.

Cuando usar un diagrama de actividad

Estos diagramas son utilizados para describir cualquier tipo de procesos. Es especialmente común para modelar gráficamente los diferentes casos de uso, transacciones o procedimientos que haya en un sistema de información. En resumen, son utilizados para representar la forma en la que un sistema hace una implementación.

La finalidad de este diagrama es modelar el workflow de una actividad a otra, pero sin tener en cuenta el paso de mensajes entre ellas. Para ello, estas actividades pueden dividirse en sistemas

por lo que una finalidad (la más común) de este diagrama puede ser capturar estos sistemas y describir como se relacionan entre sí.

También es utilizado para modelar las actividades, que podemos asemejar a requisitos funcionales de negocio, por lo que este diagrama tendrá una influencia mayor a la hora de comprender el negocio o sus funcionalidades que en la propia implementación. Hay que tener en cuenta que este diagrama ofrece una visión a alto nivel.

En resumen, algunos usos específicos podrían ser:

- Deducir los **requisitos** de negocio.
- Modelar el **flujo de trabajo** entre actividades o/y entre subsistemas.
- Comprender a alto nivel las **funcionalidades** del sistema de información.

Construcción de un diagrama de actividad

El diagrama de actividades de UML está compuesto por los siguientes elementos: **Actividades, flujos de control, nodo inicial y nodo final.**

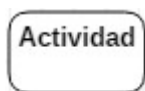
A continuación veremos qué es y como se representa cada uno de estos elementos:

Actividad

La actividad es una conducta parametrizada representada como flujo coordinado de acciones .

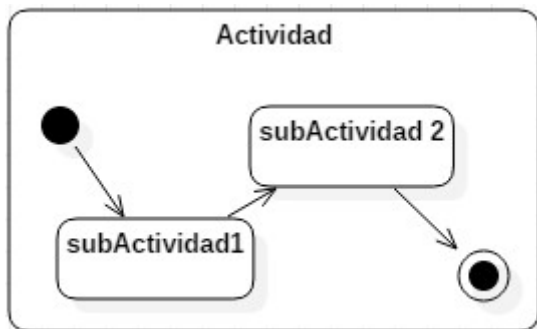
El flujo de ejecución que representa la funcionalidad deseada se modela utilizando nodos de actividad conectados por flujos de control. Un nodo puede ser la ejecución de un comportamiento subordinado, como un cálculo aritmético, una llamada a una operación o la manipulación del contenido del objeto. Los nodos de actividad también incluyen flujo de construcciones de control, como sincronización, decisión y control de concurrencia. Las actividades pueden formar jerarquías de invocación invocando otras actividades y en última instancia resolviendo acciones individuales. En un modelo orientado a objetos, las actividades generalmente se invocan indirectamente como métodos vinculados a operaciones que se invocan directamente.

Las actividades son representadas mediante un rectángulo con los bordes redondeados, que incluye en su interior el nombre de la actividad:



Notación de una actividad

También se puede representar una actividad que incluye un subdiagrama de actividades, disminuyendo de esta la granularidad de la representación, tal y como se muestra en la siguiente imagen:



Notación de una actividad compuesta

Las actividades que no tienen más descomposición se denominan acciones. Estas acciones a veces son representadas mediante pseudocódigo, aunque no es algo muy común.

Las actividades son nombradas utilizando los verbos del modelo de negocio, algunos ejemplos de estas actividades podrían ser: Buscar Objeto, Actualizar lista, Hacer pago...

Flujo entre actividades

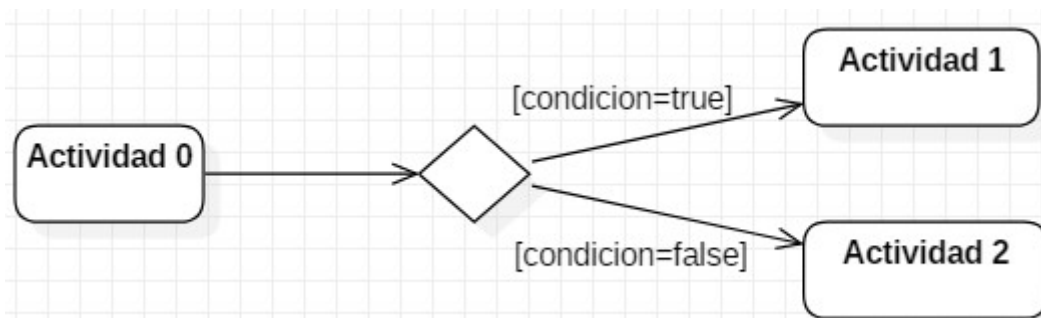
El flujo entre actividades es una clase abstracta para las conexiones dirigidas a lo largo de las cuales los tokens u objetos de datos fluyen entre los nodos de actividad. Incluye flujos de control y flujo de objetos. La fuente y el objetivo de un borde deben estar en la misma actividad que el borde.

Los flujos entre actividades se representan mediante una flecha con la punta abierta que simboliza el orden de ejecución de las actividades, a veces se incorpora un nombre en esta flecha que ayuda a que se entienda mejor:



Notación de un flujo de actividad

Estos flujos de actividades pueden usar condiciones para su actuación, estas condiciones se representan mediante un rombo, con la condición escrita entre corchetes:

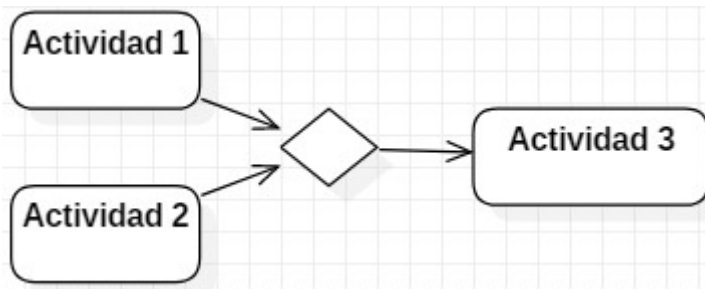


Notación de una condición

Este elemento también recibe el nombre de nodo de decisión.



El caso contrario es el llamada nodo de fusión, que recibe 2 o más flujo y emite 1:



Notación nodo de fusión

Estas dos variables pueden ser combinadas, haciendo que un nodo reciba varios flujos y, a su vez, emita también varios flujos.

Nodo inicial y nodo final

El nodo inicial es un nodo de control en el que se inicia el flujo cuando se invoca la actividad. Solo existirá uno por diagrama.

Las actividades pueden tener más de un nodo inicial. En este caso, al invocar la actividad se inician varios flujos, uno en cada nodo inicial.



Hay que tener en cuenta que los flujos también pueden comenzar en otros nodos, por lo que los nodos iniciales no son necesarios para que una actividad comience la ejecución.

Los nodos iniciales se muestran como un pequeño círculo relleno:

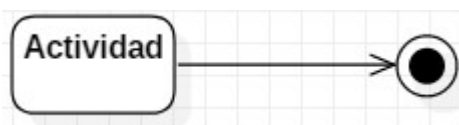


Notación de un nodo inicial

El nodo final de actividad es un nodo final de control que detiene todos los flujos en una actividad. La actividad final se introdujo en UML 2.0, hasta entonces no existía.

Una actividad puede tener más de un nodo final de actividad. El primero alcanzado detiene todos los flujos en la actividad. Un token que llega a un nodo final de actividad finaliza la actividad. En particular, detiene todas las acciones de ejecución en la actividad y destruye todos los tokens en los nodos de objetos, excepto en los nodos del parámetro de actividad de salida.

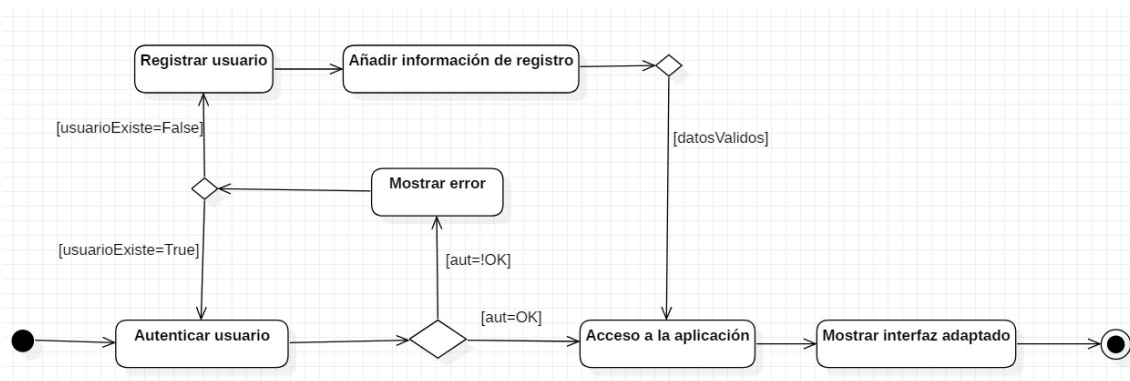
Los nodos finales de actividad se muestran como un círculo sólido con un círculo hueco dentro:



Notación de un nodo final

Ejemplo de un diagrama de actividades

A continuación se muestra el ejemplo de un diagrama de actividades para la funcionalidad de logear/registrarse un usuario en una aplicación:



Ejemplo de diagrama de actividades

Diagrama de comunicación

El **diagrama de comunicación** (denominado previamente diagrama de colaboración en las primeras versiones de UML) es un tipo de diagrama de interacción UML que muestra las interacciones entre objetos y / o partes (representadas como líneas de vida) utilizando mensajes secuenciados en una disposición de forma libre.

El diagrama de comunicación corresponde a un [diagrama de secuencia](#) simple sin mecanismos de estructuración. También se asume que el adelantamiento de mensajes (es decir, el orden de las recepciones es diferente del orden de envío de un conjunto dado de mensajes) no tendrá lugar o es irrelevante. Por lo tanto, puede verse como un diagrama de secuencia simplificado.

Las principales finalidades del diagrama de comunicación son las siguientes:

- **Modelar el paso de mensajes** entre objetos o roles que entregan las funcionalidades de casos de uso y operaciones.
- **Mostrar los mensajes** que se transmiten entre objetos y roles dentro del escenario de colaboración.
- Modelar **escenarios alternativos** dentro de casos de uso u operaciones que involucren la colaboración de diferentes objetos e interacciones.

Elementos de un diagrama de comunicación

Los diagramas de comunicación utilizan los siguientes elementos: Marco o frame, línea de vida y mensaje.

Marco o frame

Los diagramas de comunicación se pueden mostrar dentro de un **marco rectangular** con el nombre en un compartimiento en la esquina superior izquierda.

No hay un nombre específico para los tipos de encabezado de los diagramas de comunicación. Se podría utilizar la interacción de nombre de forma larga (utilizada para diagramas de interacción en general).

Línea de vida

La línea de vida es una especialización del elemento con nombre que representa a un participante individual en la interacción. Si bien las partes y las características estructurales pueden tener una multiplicidad mayor que 1, las líneas de vida representan solo una entidad que interactúa.

Si el elemento conectable al que se hace referencia tiene varios valores (es decir, tiene una multiplicidad > 1), entonces la línea de vida puede tener una expresión (selector) que especifica qué parte en particular está representada por esta línea de vida. Si se omite el selector, esto significa que se elige un representante arbitrario del elemento conectable multivalor.

Una línea de vida **se muestra como un rectángulo** (correspondiente a la «cabeza» en los diagramas de secuencia). La línea de vida en los diagramas de secuencia tiene una «cola» que representa la línea de vida, mientras que la «línea de vida» en el diagrama de comunicación no tiene línea, solo la «cabeza».

Mensaje

El mensaje en el diagrama de comunicación se muestra como una **línea con una expresión de secuencia y una flecha sobre la línea**. La flecha indica la dirección de la comunicación.

Diagrama de tiempos

El diagrama de tiempos es un diagrama UML incluido en la categoría de diagramas de interacción (perteneciente a los diagramas de comportamiento). Es utilizado para modelar el comportamiento del sistema dando especial importancia al tiempo. Los diagramas de tiempo se centran en las **condiciones que cambian** dentro y entre las líneas de vida a lo largo de un eje de tiempo lineal. Los diagramas de tiempo describen el comportamiento de los clasificadores individuales y las interacciones de los clasificadores, **enfocando la atención en el tiempo de los eventos** que causan cambios en las condiciones de las líneas de vida.

Elementos de un diagrama de tiempos

El diagrama de tiempos incluye los siguientes elementos: Líneas de vida, Estados, Restricciones de duración y Restricciones de tiempo.

Línea de vida

La línea de vida es un elemento que **representa a un participante individual** en la interacción. Si bien las partes y las características estructurales pueden tener una multiplicidad mayor que 1, las líneas de vida **representan solo una entidad que interactúa**. Sigue la misma esencia que las líneas de vida de los [diagramas de secuencia](#).

La línea de vida en los diagramas de tiempo está representada por el nombre del clasificador o la instancia que representa. Podría colocarse dentro del marco del diagrama o en un carril.

Estado

El diagrama de tiempo podría mostrar los **estados del clasificador o atributo participante**, o algunas condiciones comprobables, como un valor discreto de un atributo.

Restricción de duración

La restricción de duración es una **restricción de intervalo que se refiere a un intervalo de duración**. El intervalo de duración es la duración que se utiliza para determinar si se cumple la restricción.

La semántica de una restricción de duración se hereda de las restricciones.

Restricción de tiempo

La restricción de tiempo es una **restricción de intervalo** que se refiere a un intervalo de tiempo. El intervalo de tiempo es una expresión utilizada para determinar si se cumple la restricción.

La semántica de una restricción de tiempo se hereda de las restricciones.

La restricción de tiempo se muestra como una asociación gráfica entre un intervalo de tiempo y la construcción que restringe. Normalmente, esta asociación gráfica es una línea pequeña, por ejemplo, entre una especificación de ocurrencia y un intervalo de tiempo.

Diagrama de estados

El diagrama de máquina de estados o, más comunmente llamado, el diagrama de estados es un diagrama de comportamiento usado para especificar el comportamiento de una parte del sistema diseñado a través de transiciones de estados finitos. El formalismo de esta diagramado utilizado en UML es una variante basada en objetos de los diagramas de estado de *Harel*. Es utilizado para mostrar los estados por los que pasa un componente de un sistema de información.

El comportamiento se modela utilizando una serie de nodos que representan estados y que están conectados a través de las llamadas transiciones. Estas transiciones se activan a través de eventos.

Elementos del diagrama de estados

El diagrama de estados está formado por tres elementos: **estados, pseudoestados y transiciones**:

Estados

En el diagrama de estados, el estado modela una **situación durante la cual se cumple alguna condición invariante** (generalmente implícita). Esta situación invariante puede representar una situación estática tal como un objeto que espera que ocurra algún evento externo o cualquier otra situación. Sin embargo, también puede modelar condiciones dinámicas tales como efectuar algún tipo de comportamiento (es decir, el elemento ingresa al estado cuando el comportamiento comienza y lo deja tan pronto como se completa).

UML define los siguientes tipos de estados:

- Simple.
- Compuesto.
- De submáquina.

El primero de ellos, el estado simple, es aquel que no tiene regiones ni estados de submáquina. Se representa utilizando un rectángulo con esquinas redondeadas con el nombre del estado en el interior:

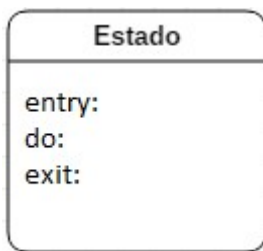


Notación de un estado

Los **estados simples** también pueden representarse utilizando dos compartimentos, uno superior y otro inferior:

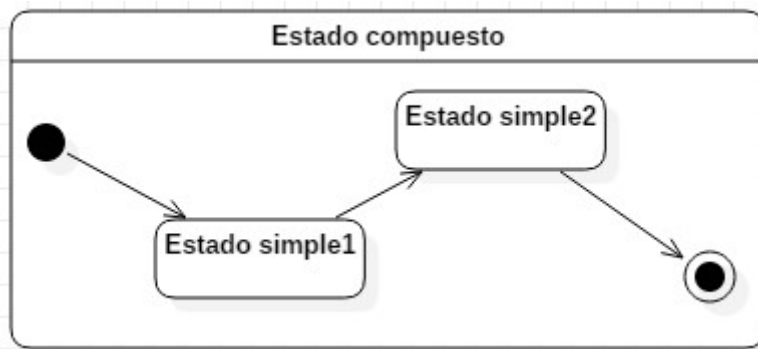
- **Superior:** Este compartimento contiene el nombre (opcional) del estado, como una cadena. Los estados sin nombres se llaman estados anónimos y se consideran estados distintos (diferentes). Los compartimentos de nombre no se deben usar si se usa una pestaña de nombre y viceversa. Se recomienda no usar el estado con el mismo nombre varias veces en el mismo diagrama.
- **Inferior:** Contiene una lista de acciones internas o actividades (comportamientos) de estado que se realizan mientras el elemento está en ese estado. La etiqueta de actividad identifica las circunstancias bajo las cuales se invocará el comportamiento especificado por la expresión de actividad. La expresión de comportamiento puede usar cualquier atributo y extremo de asociación que esté en el alcance de la entidad propietaria. Algunas etiquetas (entry, do, exit) están reservadas para propósitos especiales y no se pueden usar como nombres de eventos:
 - entry: comportamiento realizado al entrar al estado
 - do: comportamiento continuo siempre que se esté en el estado.
 - exit: comportamiento realizado al salir del estado.

Su representación es la siguiente:



Notación de un estado 2

El estado compuesto es aquel que tiene subestados, se representa con un subdiagrama de estados en su interior:



Notación de un estado

compuesto

Pseudoestados

Un pseudoestado es un vértice abstracto que abarca diferentes tipos de vértices transitorios en el gráfico de máquina de estado, siendo los más comunes los pseudoestados inicial y final.

Los pseudoestados se usan generalmente para **conectar múltiples transiciones** en rutas de transición de estado más complejas.



El **pseudoestado inicial** representa el comienzo de un diagrama, solo puede existir uno por diagrama. Se representa a través de un pequeño círculo relleno:



Notación pseudoestado inicial

El **pseudoestado final** representa el final del diagrama, al contrario que el inicial **pueden existir varios pseudoestados finales**. Indica que se ha finalizado la máquina de estados. Se representa con un círculo que rodea un círculo negro relleno:



Notación estado final

Transición

Una transición es una **relación dirigida entre un estado de origen y un estado de destino**. Puede ser parte de un estado compuesto. Se representa con una flecha que va desde el estado origen al estado destino. En ocasiones, si es relevante, se incluye una notación que indica la condición desencadenante para cambiar de estado.

Diagrama de perfiles

Un diagrama de perfiles permite extender UML para su uso con **una plataforma de programación en particular** (como el framework .NET de Microsoft o la plataforma Java Enterprise Edition), o modelar sistemas destinados a ser usados en un dominio en particular (por ejemplo, medicina, servicios financieros o ingeniería especializada).

Las herramientas de generación de código pueden usar perfiles para generar código dirigido a una plataforma o entorno específico (la forma precisa en que se implementan los perfiles depende en cierto modo de la herramienta de modelado UML utilizada).

Para comprender cómo funcionan los perfiles y el diagrama de perfiles es necesario comprender la naturaleza fundamental de UML, ya que se trata de un **metamodelo**. La palabra modelo implica una abstracción de algún sistema del mundo real. La palabra meta implica una abstracción adicional de cualquier concepto al que lo apliquemos. La palabra metadatos se usa a menudo en el contexto de una base de datos, por ejemplo, y en términos simples se entiende que hace referencia a datos sobre los datos. Por lo tanto, un metamodelo (como podrás deducir) describe las propiedades de un modelo, es como un modelo de un modelo. **Un modelo está limitado por su metamodelo** de la misma manera que un software está limitado por la gramática y la sintaxis del lenguaje de programación que ha sido utilizado a la hora de escribirlo. **Cada elemento del Lenguaje de modelado unificado está definido por una metaclase incluida en el metamodelo.**

Elementos de un diagrama de perfiles

Los nodos y elementos gráficos utilizados en los diagramas de perfil son: **perfil, metaclase, estereotipo, extensión, referencia y aplicación de perfil.**

Perfil

Un perfil es un paquete que extiende a un metamodelo de referencia (en este caso podría ser UML) permitiendo adaptar el metamodelo con directrices que son específicas de un dominio, plataforma o método de desarrollo de software en particular. En otras palabras, **el perfil es un mecanismo de extensión ligero al estándar UML.**

Un perfil introduce varias restricciones en el metamodelo ordinario mediante el uso de las metaclases definidas en este paquete. La construcción de la extensión primaria es el estereotipo, que se define como parte del perfil y extiende algún elemento de la metaclase. No es posible definir un perfil independiente, sin su metamodelo de referencia.

Un perfil utiliza la misma notación que un paquete del [diagrama de paquetes](#), con la adición de que la palabra clave «perfil» se muestra antes o encima del nombre del paquete.

Metaclase

Una metaclase es una clase de perfiles y un elemento empaquetable que puede extenderse a través de uno o más estereotipos.

Se puede mostrar una metaclase con el estereotipo opcional «Metaclase» que se muestra arriba o antes de su nombre (toda la «metaclase» en minúsculas se usó en versiones UML anteriores a 2.4).

La metaclass **puede extenderse por uno o más estereotipos** utilizando un tipo especial de asociación: extensión.

Estereotipo

Un estereotipo es una **clase de perfil** que define cómo una metaclass existente puede extenderse como parte de un perfil. Permite el uso de una plataforma o una terminología específica del dominio o una notación en lugar de, o además de, los utilizados para la metaclass extendida.

Un estereotipo no puede usarse solo, sino que siempre debe usarse con una de las metaclasses que extiende. El estereotipo no puede extenderse por otro estereotipo.

Un estereotipo usa la misma notación que una clase, con la palabra clave «estereotipo» mostrada antes o encima del nombre del estereotipo. Los nombres de estereotipos no deben coincidir con los nombres de palabras clave para el elemento del modelo extendido.

Dado que el estereotipo es una clase, puede tener propiedades. Las propiedades de un estereotipo se conocen como definiciones de etiqueta. Cuando se aplica un estereotipo a un elemento del modelo, los valores de las propiedades se denominan valores etiquetados.

Extensión

Una extensión es la **relación de asociación** que se usa para indicar que las propiedades de una metaclass se extienden a través de un estereotipo, y brinda la posibilidad de agregar flexiblemente los estereotipos a las clases y eliminarlas más adelante, si es necesario.

Un extremo de la asociación de extensión es una propiedad ordinaria y el otro extremo es un extremo de extensión. La propiedad vincula la extensión a una metaclass, mientras que el extremo de la extensión vincula la extensión al estereotipo que extiende la metaclass.

El extremo de la extensión es un extremo navegable, propiedad de la extensión. Esto permite que una instancia de estereotipo se adjunte a una instancia del clasificador extendido sin agregar una propiedad al clasificador.



La notación para una extensión es una **flecha con la punta de flecha de triángulo relleno** que apunta desde un estereotipo a la metaclass extendida.

La extensión es una asociación que indica que las propiedades de una metaclass se extienden a través de un estereotipo.

Una extensión no requerida significa que una instancia de un estereotipo puede vincularse a una instancia de una metaclass extendida a voluntad, y también eliminarse posteriormente a voluntad. Sin embargo, no es necesario que cada instancia de una metaclass sea extendida. Una instancia de un estereotipo se elimina cuando se elimina la instancia de la metaclass extendida o cuando el perfil que define el estereotipo se elimina de los perfiles aplicados del paquete.

Una extensión requerida significa que una instancia de un estereotipo **siempre debe estar vinculada** a una instancia de la metaclass extendida. La instancia del estereotipo generalmente se elimina solo cuando se elimina la instancia de la metaclass extendida o cuando el perfil que define el estereotipo se elimina de los perfiles aplicados del paquete.

Para que una extensión sea obligatoriamente requerida se utiliza la propiedad {required}.

Referencia

La referencia es una **relación de importación** representada por la importación del elemento «metaclassReference» y la importación del paquete «metamodelReference».

Las importaciones de elementos «metaclassReference» y las importaciones de paquetes «metamodelReference» sirven para dos propósitos:

- **Identificar los elementos del metamodelo** de referencia que importa el perfil.
- **Especificar las reglas de filtrado del perfil.** Las reglas de filtrado determinan qué elementos del metamodelo son visibles cuando se aplica el perfil y cuáles están ocultos.

Hay que tener en cuenta que la aplicación de un perfil no cambia el modelo subyacente de ninguna manera; simplemente define una vista del modelo subyacente. En general, solo los elementos del modelo que son instancias de metaclases de referencia importadas serán visibles cuando se aplique el perfil. De forma predeterminada, los elementos del modelo cuyas metaclases son públicas y son propiedad del metamodelo de referencia son visibles.

Aplicación de perfil

La aplicación de perfil es una **relación dirigida** que se utiliza para mostrar qué perfiles se han aplicado a un paquete.

Se pueden aplicar uno o más perfiles a un paquete que se crea a partir del mismo metamodelo que se extiende por el perfil. Aplicar un perfil significa que está **permitido, pero no necesariamente requerido**, aplicar los estereotipos que se definen como parte del perfil.

Es posible aplicar múltiples perfiles a un paquete siempre que no tengan restricciones en conflicto. Si un perfil que se está aplicando depende de otros perfiles, entonces esos perfiles deben aplicarse primero.

Cuando se aplica un perfil, se deben crear instancias de los estereotipos apropiados para aquellos elementos que son instancias de metaclases con extensiones requeridas. El modelo no está bien formado sin estos casos.



Una vez que se ha aplicado un perfil a un paquete, se le permite eliminar el perfil aplicado a voluntad. Eliminar un perfil implica que todos los elementos que son instancias de elementos definidos en un perfil se eliminan. Un perfil que se ha aplicado no puede eliminarse a menos que otros perfiles aplicados que dependen de él se eliminen primero.

El perfil aplicado se muestra mediante una flecha discontinua con una punta de flecha abierta desde el paquete al perfil aplicado con la palabra clave «apply» cerca de la flecha.